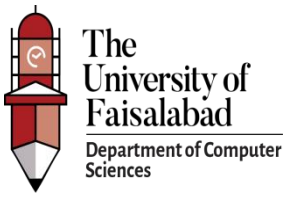


The University of Faisalabad

Department of Computational Sciences

BSAI 6th Semester

LAB MANUAL

Course Title: Data Mining **Course Code:** DS-303

Name : Ayesha Imran

Registration no : 2023-BS-AI-011

Submitted to : Sir Arham

Degree : Bachelors of Artificial Intelligence

Section A

Batch : 2023-2027

TABLE OF CONTENT

LAB-1

Knime Introduction

Introduction to Data Mining

Data Mining is the process of extracting useful information, patterns, and knowledge from large amounts of data. It is an important part of the field of Data Science and helps organizations make better decisions.

In simple words, data mining means analyzing data to discover hidden patterns and relationships that are not easily visible.

Why Data Mining is Important

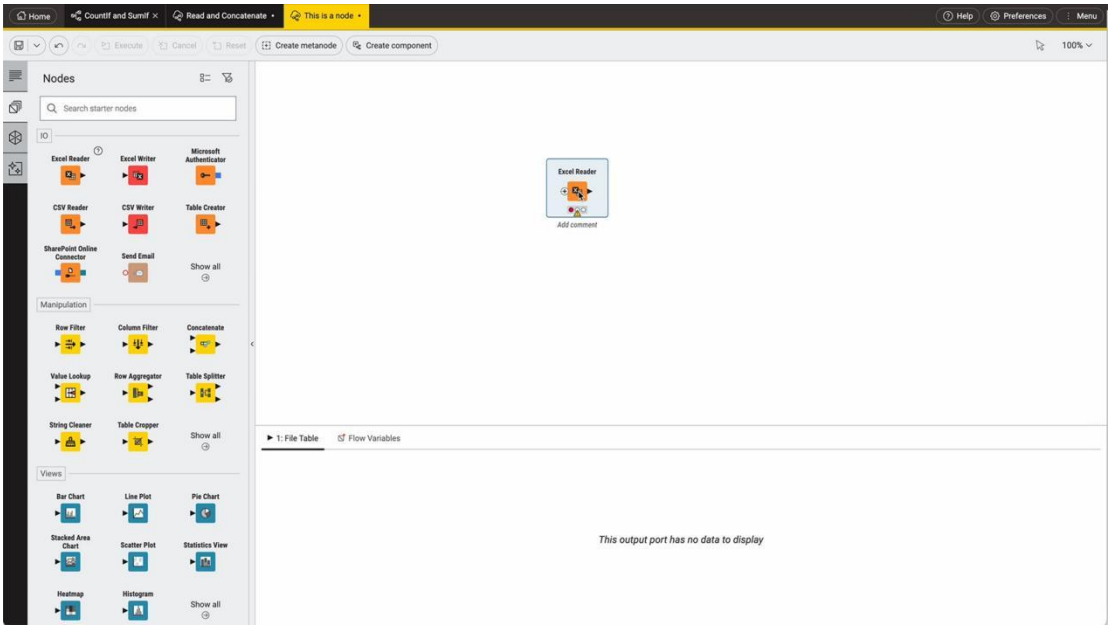
Today, huge amounts of data are generated every day. Data mining helps to:

- Identify trends and patterns
- Improve decision making
- Predict future outcomes
- Understand customer behavior

Data mining process:

Introduction to software :

KNIME (Konstanz Information Miner) is an open-source data analytics platform used for data mining, data analysis, and machine learning. It provides a user-friendly, graphical interface where users can create workflows by connecting different nodes instead of writing code. **KNIME** allows users to easily perform tasks such as data preprocessing, visualization, and model building, making it suitable for both beginners and professionals. It is widely used in business intelligence and data science for analyzing large datasets and extracting meaningful insights.



Dataset Overview: Retail Sales Dataset

https://www.kaggle.com/datasets/mohammadtalib786/retail-sales-dataset

This dataset contains transactional retail data that captures customer purchases across different countries. Each record represents a single item purchased within a transaction.

Key Attributes (Columns)

The dataset includes the following main features:

- **UserId** → Unique identifier for each customer
- **TransactionId** → Unique ID for each transaction
- **TransactionTime** → Date and time when the purchase was made
- **ItemCode** → Unique code for each product
- **ItemDescription** → Name/description of the product
- **NumberOfItemsPurchased** → Quantity of items bought in a transaction
- **CostPerItem** → Price of a single item
- **Country** → Country where the transaction occurred

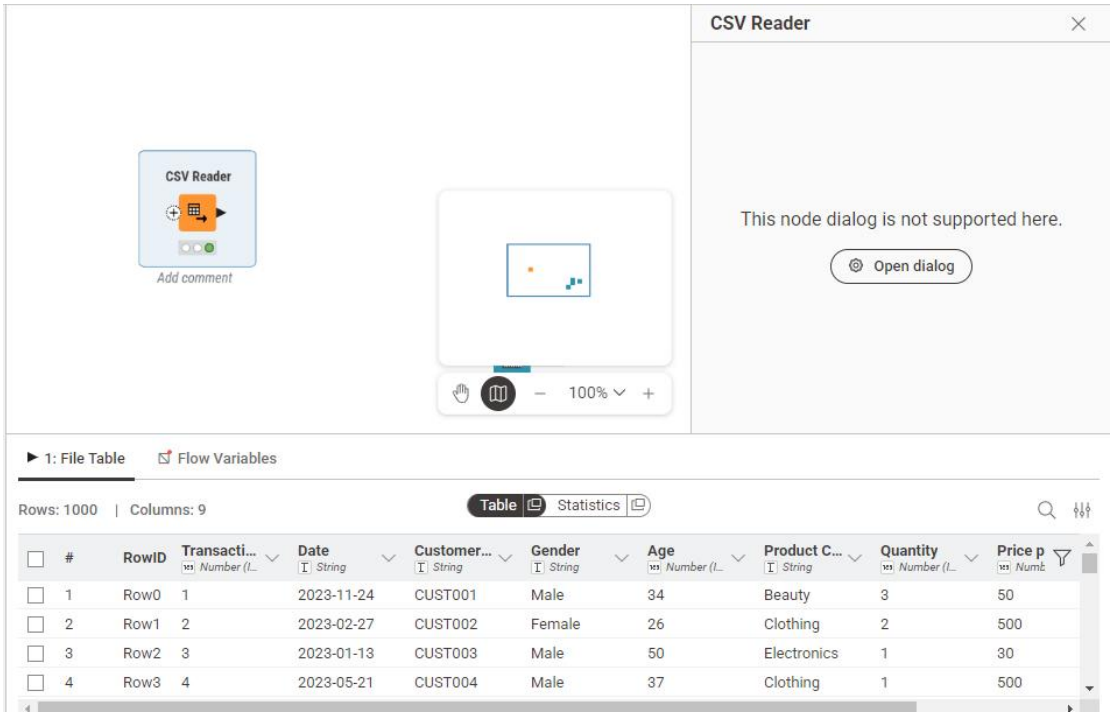
What This Dataset Represents

This dataset reflects:

- Customer buying behavior
- Product demand patterns
- Sales distribution across countries

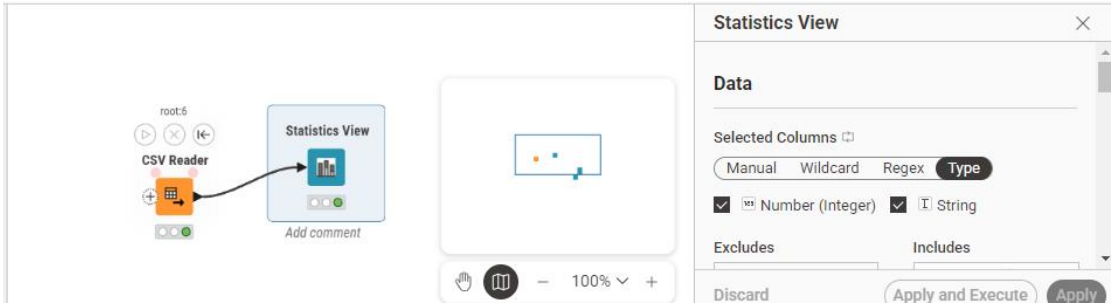
Each row corresponds to a **single product within a transaction**, meaning one transaction may have multiple rows.

1. Import Dataset in KNIME



#	RowID	Transacti... Number (I...	Date String	Customer... String	Gender String	Age Number (I...	Product C... String	Quantity Number (I...	Price p Numb
1	Row0	1	2023-11-24	CUST001	Male	34	Beauty	3	50
2	Row1	2	2023-02-27	CUST002	Female	26	Clothing	2	500
3	Row2	3	2023-01-13	CUST003	Male	50	Electronics	1	30
4	Row3	4	2023-05-21	CUST004	Male	37	Clothing	1	500

2. Identify Data Types

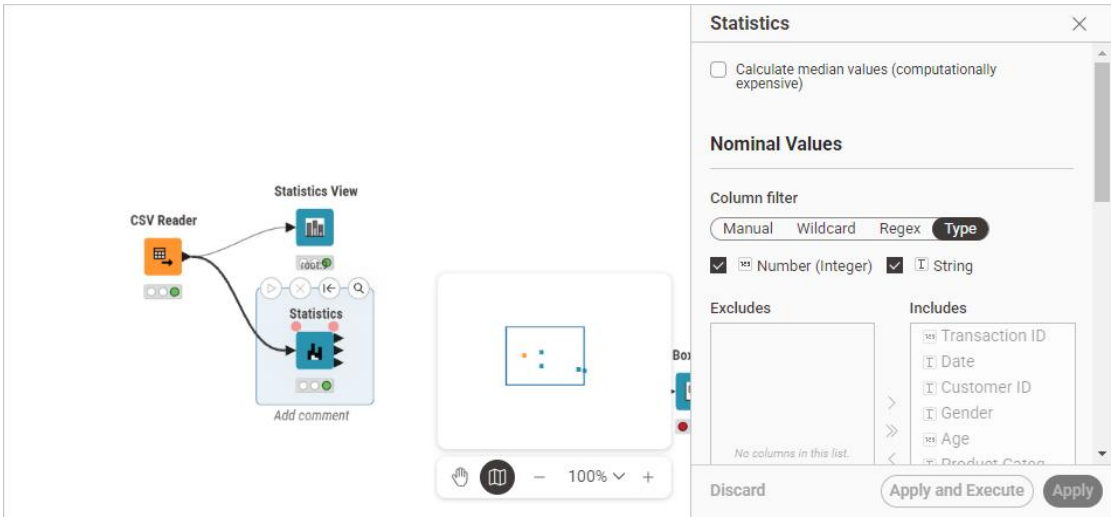


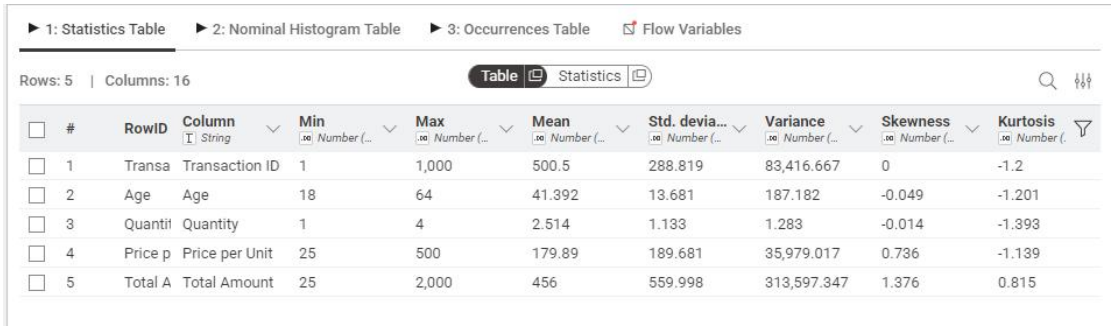
Statistics

Rows: 9 | Columns: 2

Name	Type
Transaction ID	Number (Integer)
Date	String
Customer ID	String
Gender	String
Age	Number (Integer)
Product Category	String
Quantity	Number (Integer)
Price per Unit	Number (Integer)
Total Amount	Number (Integer)

3. Generate Summary Statistics



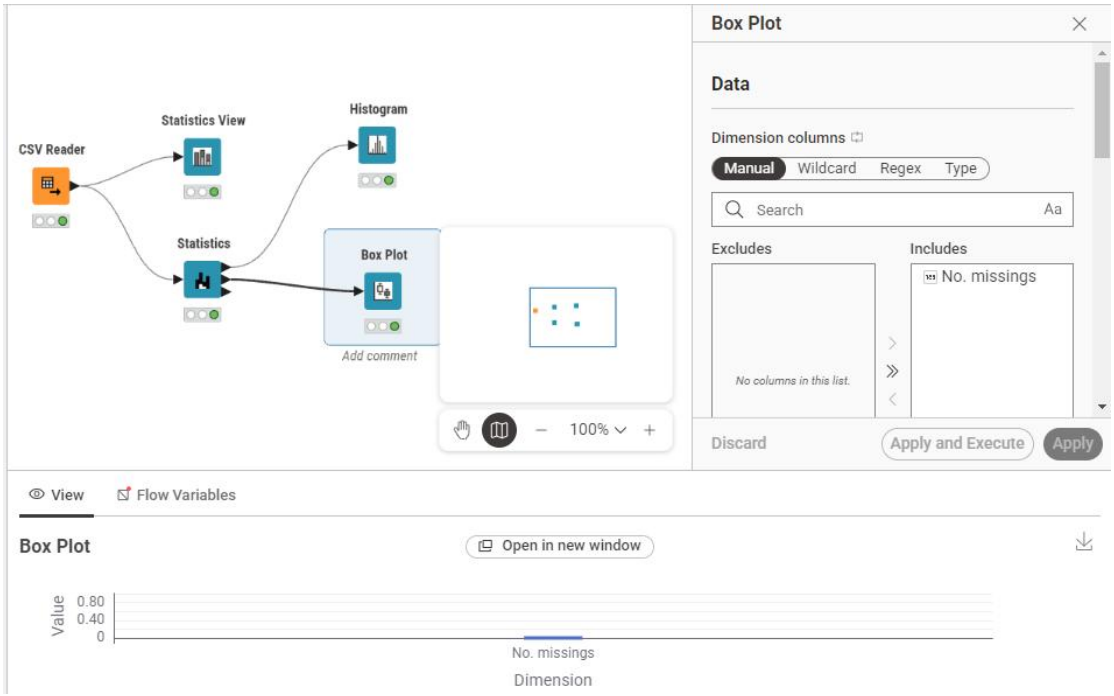


#	RowID	Column	Min	Max	Mean	Std. devia...	Variance	Skewness	Kurtosis
		String	Number (...	Number (...	Number (...	Number (...	Number (...	Number (...	Number (
1	Transa	Transaction ID	1	1,000	500.5	288.819	83,416.667	0	-1.2
2	Age	Age	18	64	41.392	13.681	187.182	-0.049	-1.201
3	Quantit	Quantity	1	4	2.514	1.133	1.283	-0.014	-1.393
4	Price p	Price per Unit	25	500	179.89	189.681	35,979.017	0.736	-1.139
5	Total A	Total Amount	25	2,000	456	559.998	313,597.347	1.376	0.815

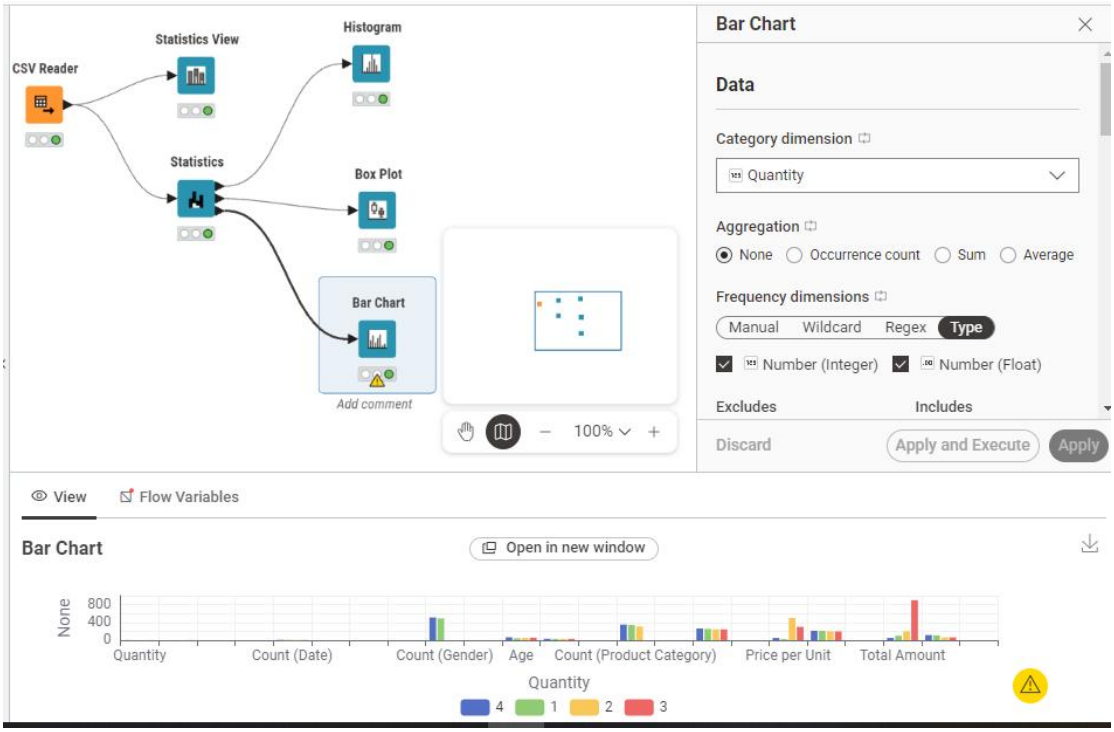
4. Explore Attributes (Data Analysis)

- **Histogram (Numeric Data)**

- **Box Plot**



- **Bar Chart**



LAB-2

CRISP-DM

Dataset : https://www.kaggle.com/datasets/sarahashmi/churn-modeling-csv

Types of Data

Data can be classified into different types based on structure and format:

1. Structured Data

- Organized in tables (rows & columns)
- Easy to store and analyze
- Example: databases, Excel sheets

2. Unstructured Data

- No fixed format
- Harder to analyze
- Example: text, images, videos, audio

3. Semi-Structured Data

- Not fully structured but has some organization
- Example: JSON, XML files

Other Common Data Types:

- **Numerical Data**
    - Numbers (e.g., age, salary)
    - Can be discrete or continuous
- **Categorical Data**
    - Labels or categories
    - Example: gender, color, yes/no
- **Time-Series Data**
    - Data over time
    - Example: stock prices, temperature

Real-World Data Mining Applications

Data mining is widely used in many fields:

1. Business & Marketing

- Customer segmentation
- Sales prediction
- Targeted advertising

2. Healthcare

- Disease prediction
- Patient diagnosis
- Medical image analysis

3. Banking & Finance

- Fraud detection
- Credit scoring
- Risk analysis

4. E-commerce

- Product recommendation systems
- Customer behavior analysis

5. Education

- Student performance prediction
- Personalized learning systems

6. Social Media

- Sentiment analysis
- Trend detection

CRISP-DM Phases in Orange (with practical steps)

CRISP-DM has 6 phases.

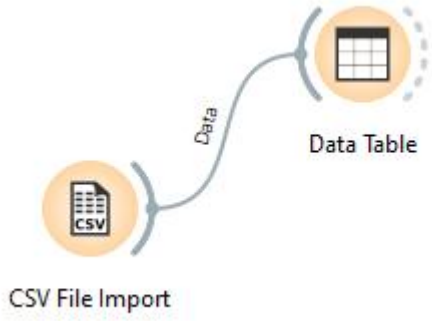
1. Business Understanding

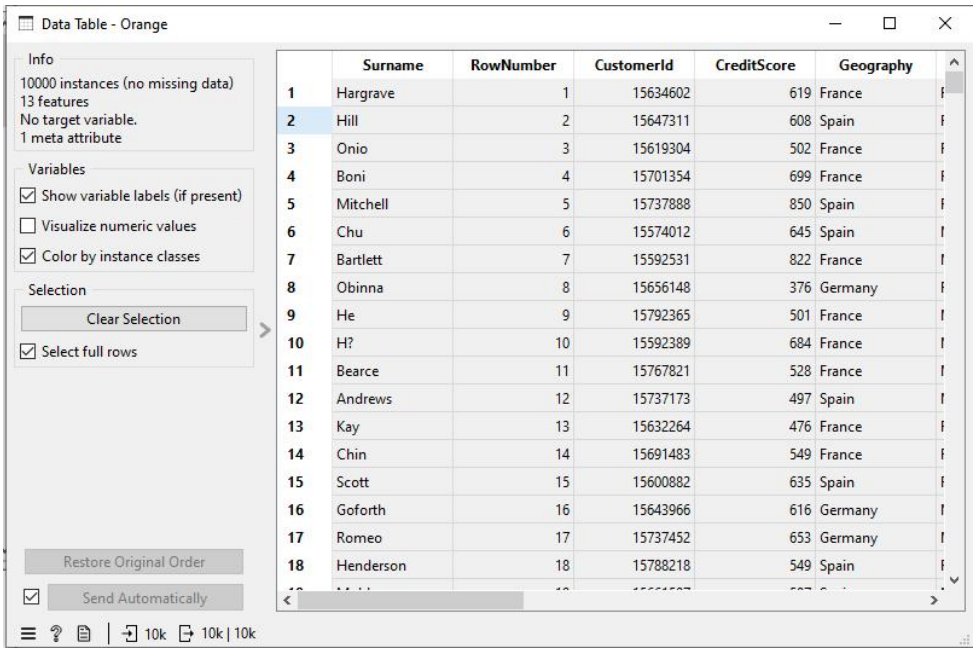
Goal: Predict customer churn (Yes/No)

- Problem: Which customers will leave the bank?
- Target variable: **Exited (Churn)**

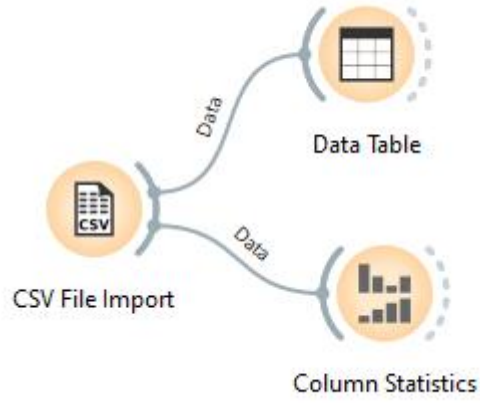
2. Data Understanding

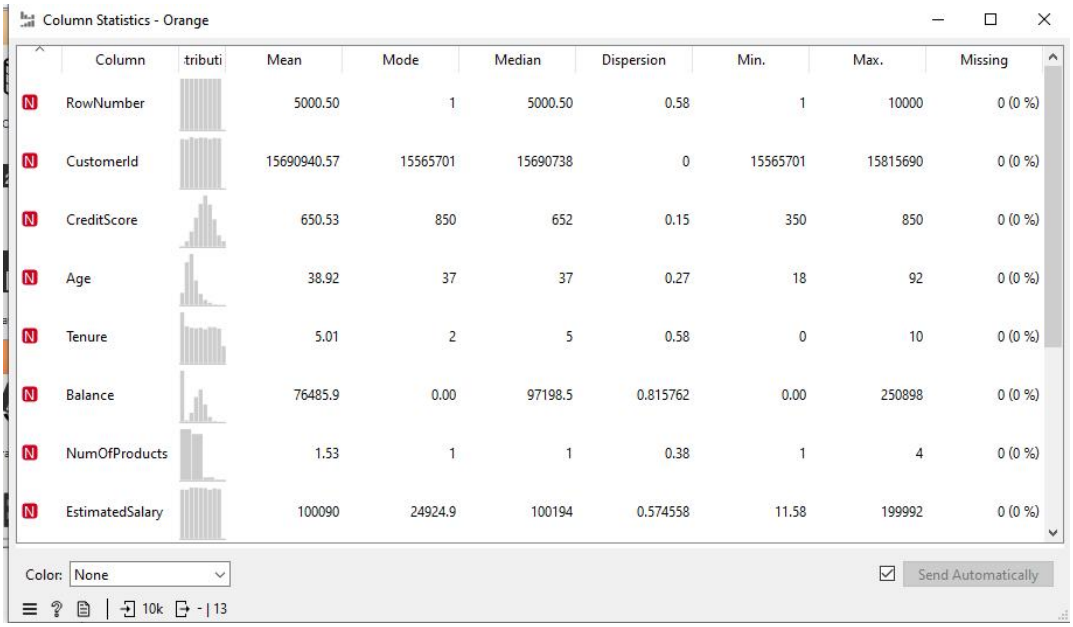
- Collect and explore data
- Check data quality and patterns



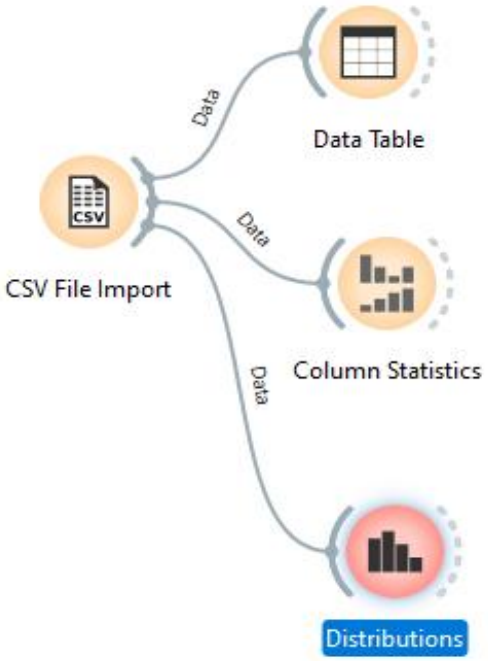


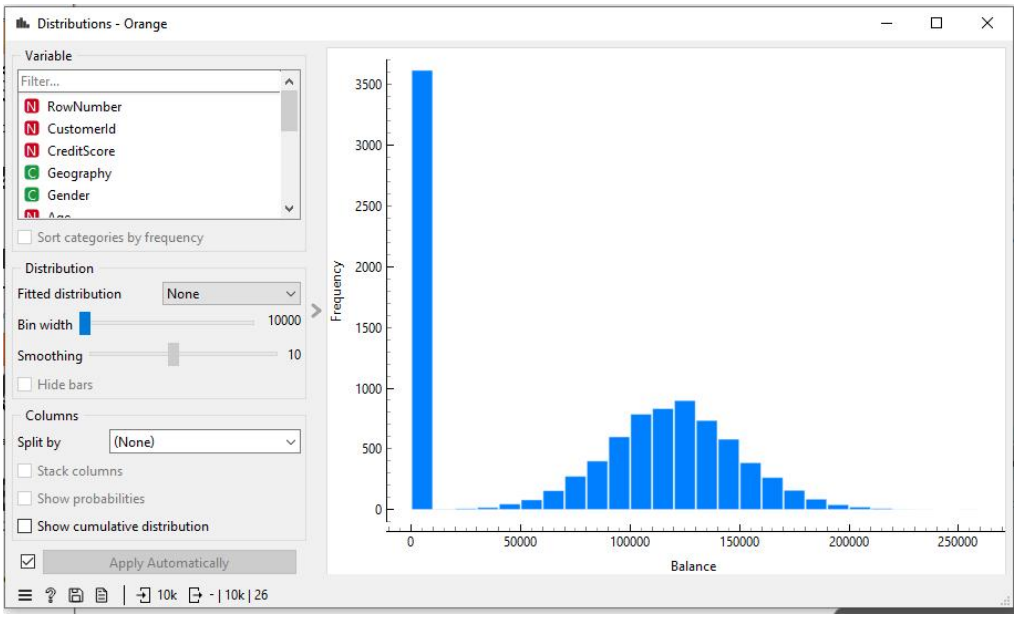
	Surname	RowNumber	CustomerId	CreditScore	Geography
1	Hargrave	1	15634602	619	France
2	Hill	2	15647311	608	Spain
3	Onio	3	15619304	502	France
4	Boni	4	15701354	699	France
5	Mitchell	5	15737888	850	Spain
6	Chu	6	15574012	645	Spain
7	Bartlett	7	15592531	822	France
8	Obinna	8	15656148	376	Germany
9	He	9	15792365	501	France
10	H?	10	15592389	684	France
11	Bearce	11	15767821	528	France
12	Andrews	12	15737173	497	Spain
13	Kay	13	15632264	476	France
14	Chin	14	15691483	549	France
15	Scott	15	15600882	635	Spain
16	Goforth	16	15643966	616	Germany
17	Romeo	17	15737452	653	Germany
18	Henderson	18	15788218	549	Spain



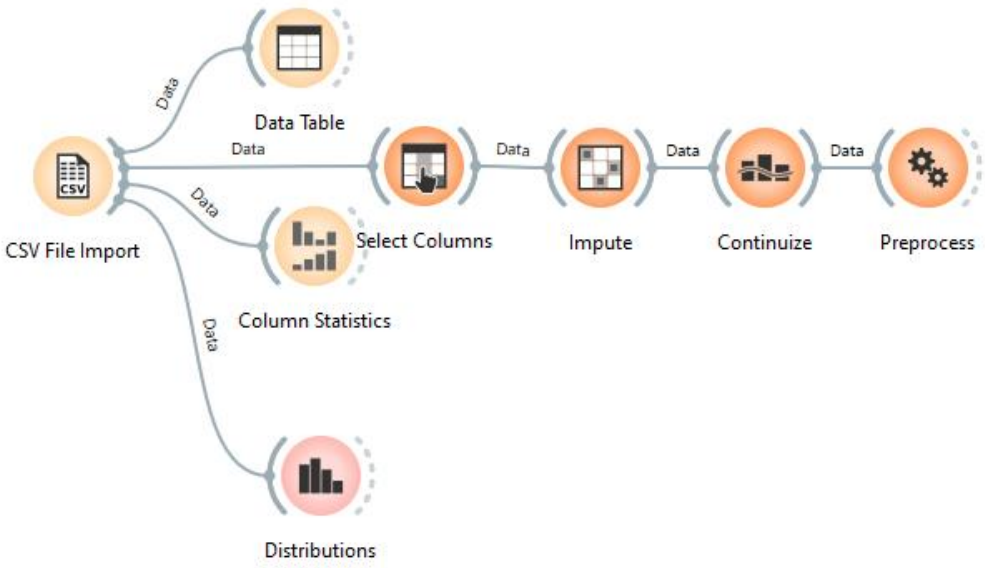


Column	tributi	Mean	Mode	Median	Dispersion	Min.	Max.	Missing
RowNumber		5000.50	1	5000.50	0.58	1	10000	0 (0 %)
CustomerId		15690940.57	15565701	15690738	0	15565701	15815690	0 (0 %)
CreditScore		650.53	850	652	0.15	350	850	0 (0 %)
Age		38.92	37	37	0.27	18	92	0 (0 %)
Tenure		5.01	2	5	0.58	0	10	0 (0 %)
Balance		76485.9	0.00	97198.5	0.815762	0.00	250898	0 (0 %)
NumOfProducts		1.53	1	1	0.38	1	4	0 (0 %)
EstimatedSalary		100090	24924.9	100194	0.574558	11.58	199992	0 (0 %)





3. Data Preparation



Remove useless columns : use select columns node

Handle missing values : Use Impute

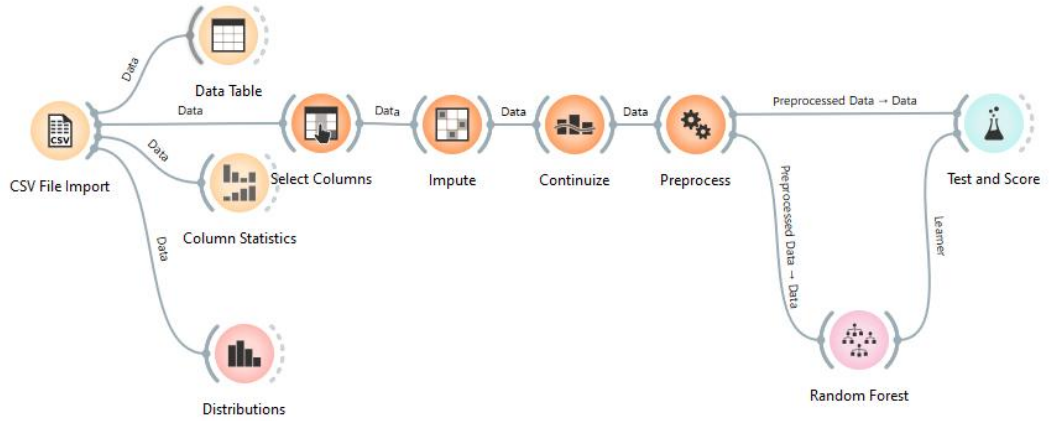
Convert categorical data : Use Continuize

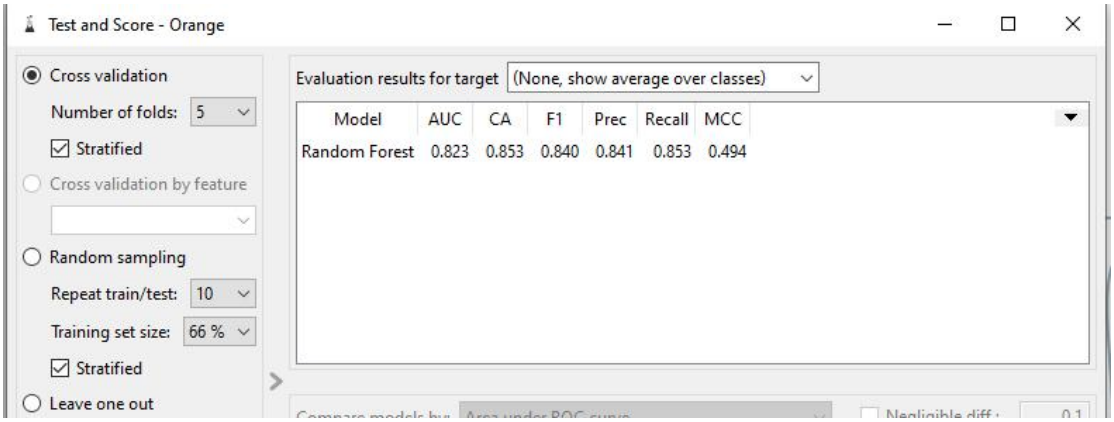
Normalize data : Use preprocess

4. Modeling

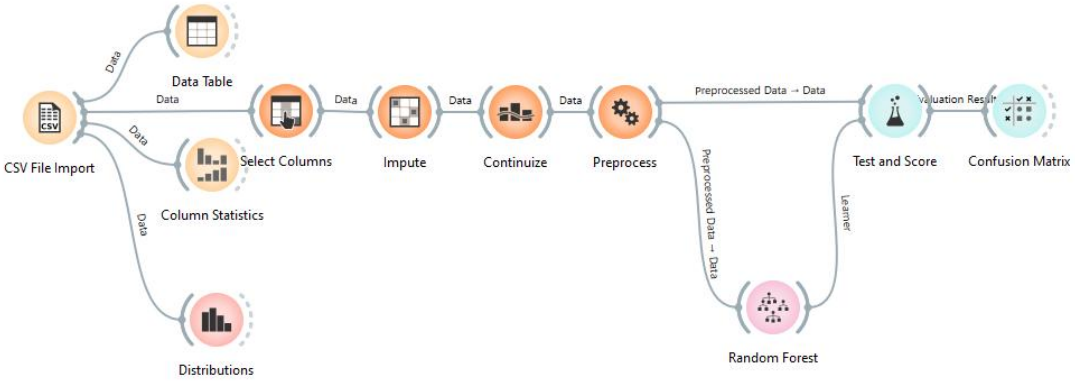
5. Evaluation

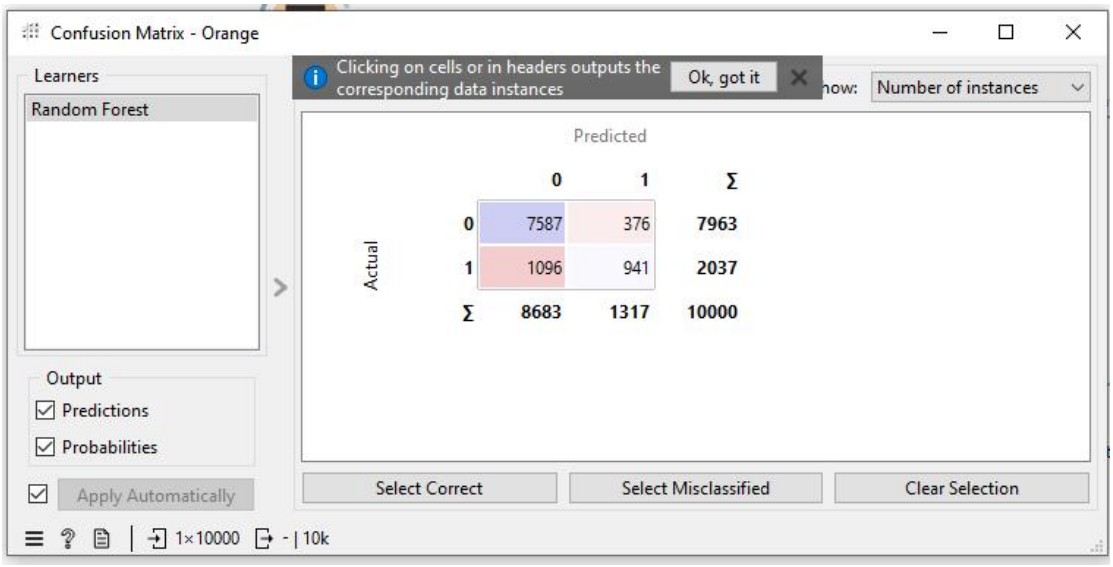
- **Test & Score**



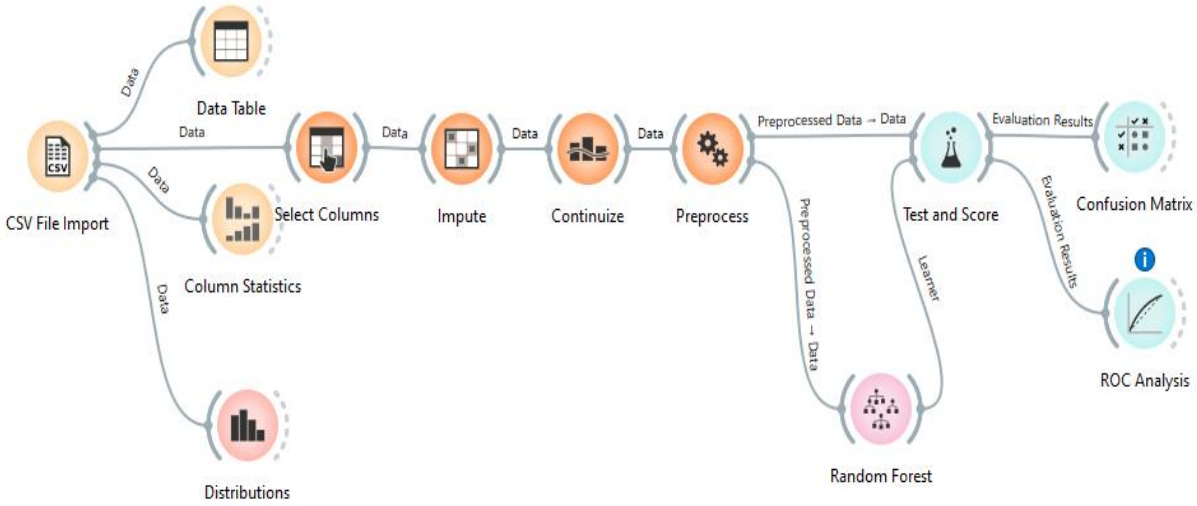


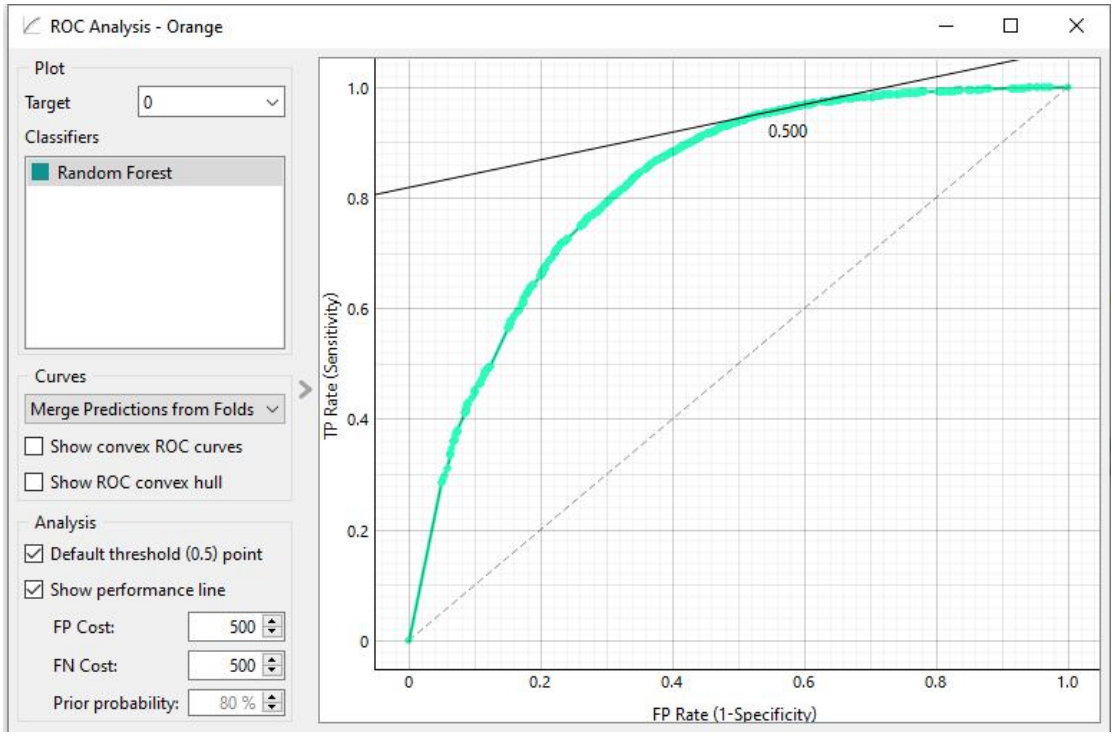
- **Confusion Matrix**



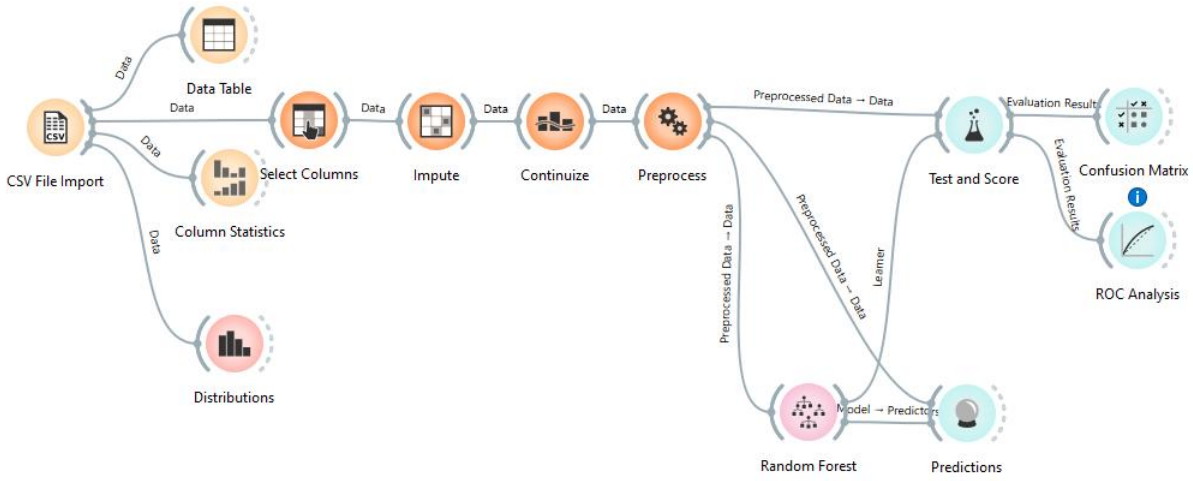


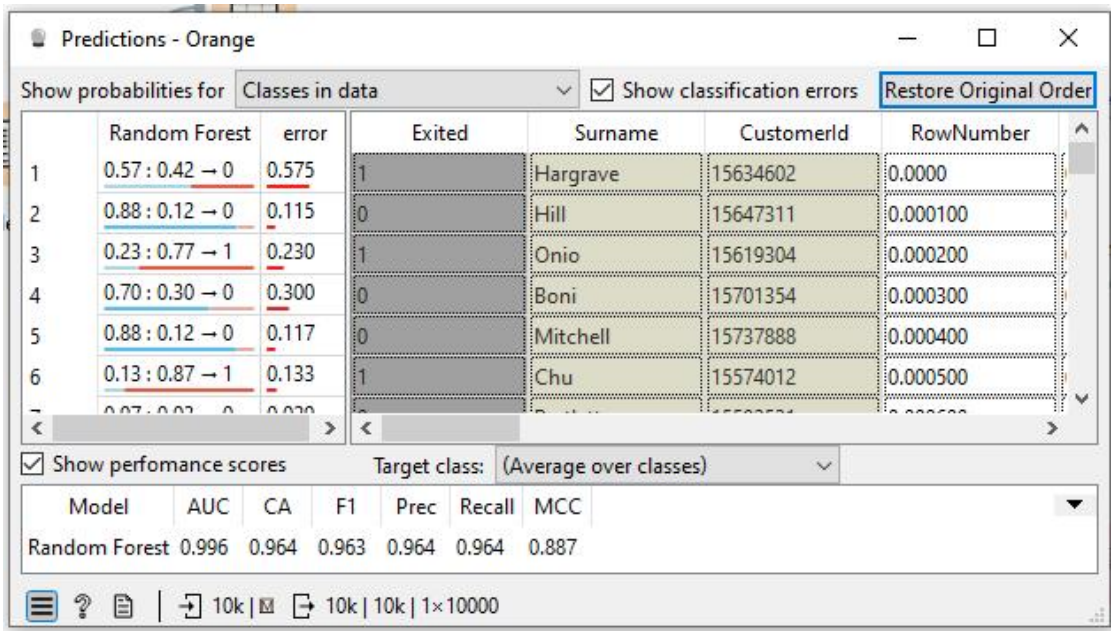
- **ROC Analysis**





6. Deployment




Predictions - Orange
Show probabilities for
Classes in data
Show classification errors
Restore Original Order
Random Forest
error
Exited
Surname
CustomerId
RowNumber
1 0.57 : 0.42 → 0 0.575 1 Hargrave 15634602 0.0000
2 0.88 : 0.12 → 0 0.115 0 Hill 15647311 0.000100
3 0.23 : 0.77 → 1 0.230 1 Onio 15619304 0.000200
4 0.70 : 0.30 → 0 0.300 0 Boni 15701354 0.000300
5 0.88 : 0.12 → 0 0.117 0 Mitchell 15737888 0.000400
6 0.13 : 0.87 → 1 0.133 1 Chu 15574012 0.000500
Show perfomance scores
Target class:
(Average over classes)
Model AUC CA F1 Prec Recall MCC
Random Forest 0.996 0.964 0.963 0.964 0.964 0.887
10k | 10k | 10k | 1×10000

LAB-3

Data Cleaning

Dataset: https://www.kaggle.com/datasets/aranyogeshm/healthcare-dataset

Handling Missing Values

Missing data occurs when no value is stored for a specific attribute in a record. This can happen due to equipment malfunction, human error, or simply because the information was not available at the time of entry.

Removing Duplicates

Duplicate data—where multiple records represent the same real-world entity—can significantly skew statistical analysis and lead to over-represented patterns. This often happens during **data integration**, where information from multiple databases is merged.

Noise Filtering

Noise refers to a random error or variance in a measured variable. While missing values are "empty," noisy data is "incorrect" or "distorted" (e.g., a person's age listed as -5 or 200). Noise filtering aims to smooth out these fluctuations to reveal the underlying trend.

Step 1: Import libraries

```
[1]: import pandas as pd
     import numpy as np
```

Step 2: Load dataset

```
[2]: df = pd.read_csv("Iris.csv")

     print("Original Shape:", df.shape)
     Original Shape: (150, 6)
```

Step 3: Handle Missing Values

[5]:
```
missing_before = df.isnull().sum().sum()

# Fill numeric columns with mean
for col in df.select_dtypes(include=np.number):
    df[col] = df[col].fillna(df[col].mean())

# Fill text columns with mode
for col in df.select_dtypes(include='object'):
    df[col] = df[col].fillna(df[col].mode()[0])

missing_after = df.isnull().sum().sum()
```

Step 4: Remove Duplicates

[6]:
```
duplicates_before = df.duplicated().sum()

df = df.drop_duplicates()

duplicates_after = df.duplicated().sum()
```

Step 5: Noise Filtering

[7]:
```
# Remove outliers using simple IQR method
for col in df.select_dtypes(include=np.number):
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df = df[(df[col] >= lower) & (df[col] <= upper)]

# Clean text (lowercase + strip spaces)
for col in df.select_dtypes(include='object'):
    df[col] = df[col].str.lower().str.strip()
```

Step 6: Calculate KPIs

[8]:
```
# % Missing Reduced
missing_reduced_percent = ((missing_before - missing_after) / missing_before) * 100 if missing_before != 0 else 0

# Duplicate Removal %
duplicate_removed_percent = ((duplicates_before - duplicates_after) / duplicates_before) * 100 if duplicates_before != 0 else 0

# Simple Data Quality Score (custom)
total_cells = df.shape[0] * df.shape[1]
missing_now = df.isnull().sum().sum()

data_quality_score = 100 - ((missing_now / total_cells) * 100)
```

Step 7: Output Results

[9]:
```
print("\n--- KPIs ---")
print("Missing Reduced (%):", round(missing_reduced_percent, 2))
print("Duplicate Removal (%):", round(duplicate_removed_percent, 2))
print("Data Quality Score (%):", round(data_quality_score, 2))

print("\nCleaned Shape:", df.shape)
```

```
--- KPIs ---
Missing Reduced (%): 0
Duplicate Removal (%): 0
Data Quality Score (%): 100.0

Cleaned Shape: (146, 6)
```

LAB-4

Normalization, standardization, discretization, summary stats

Dataset : https://www.kaggle.com/datasets/mosaadhendam/loan-prediction-dataset

1. Summary Statistics

Summary statistics provide a quick overview of the central tendency, dispersion, and shape of the dataset's distribution.

```
[1]: import pandas as pd

     # Load the dataset
     df = pd.read_csv('loan_prediction_dataset.csv')
```

Display summary statistics for numerical columns

```
[9]: print("Summary Statistics:")
     display(df.describe())
```

Summary Statistics:

	Age	Income	Credit_Score	Loan_Amount	Loan_Term	Loan_Approved
count	2000.000000	2000.000000	2000.000000	2000.000000	2000.00000	2000.000000
mean	43.805500	84533.585000	577.055000	25460.315000	35.47800	0.171000
std	14.929203	37771.169751	157.525951	14116.737774	16.98587	0.376603
min	18.000000	20155.000000	300.000000	1060.000000	12.00000	0.000000
25%	31.000000	50925.250000	440.000000	13444.250000	24.00000	0.000000
50%	44.000000	84073.500000	578.500000	25446.000000	36.00000	0.000000
75%	56.000000	117523.250000	715.250000	37949.250000	48.00000	0.000000
max	69.000000	149992.000000	849.000000	49994.000000	60.00000	1.000000

Display summary for categorical columns

```
[10]: print("\nSummary Statistics for Categorical Columns:")
      display(df.describe(include=['object']))
```

Summary Statistics for Categorical Columns:

	Employment_Status
count	2000
unique	3
top	Employed
freq	1260

2. Normalization (Min-Max Scaling)

Normalization rescales the data into a specific range, typically $[0, 1]$. This is useful when the features have different scales and the algorithm (like K-NN or Neural Networks) is sensitive to the magnitude of values.

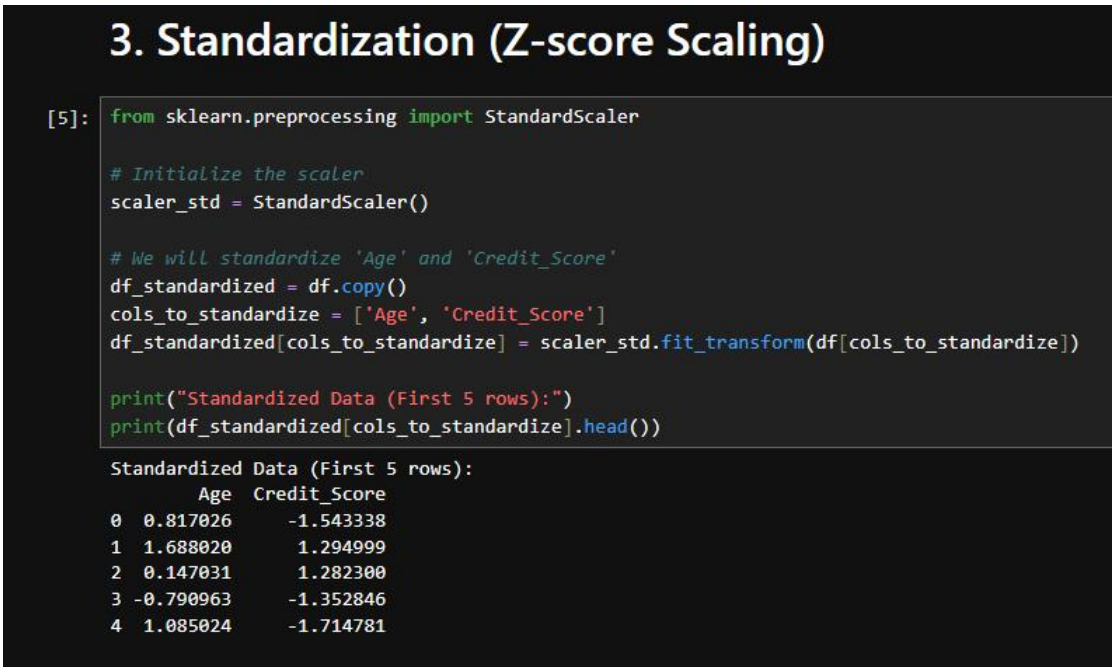
2. Normalization (Min-Max Scaling)

```
[4]: from sklearn.preprocessing import MinMaxScaler

     # Initialize the scaler
     scaler_minmax = MinMaxScaler()

     # We will normalize 'Income' and 'Loan_Amount'
     df_normalized = df.copy()
     cols_to_normalize = ['Income', 'Loan_Amount']
     df_normalized[cols_to_normalize] = scaler_minmax.fit_transform(df[cols_to_normalize])

     print("Normalized Data (First 5 rows):")
     print(df_normalized[cols_to_normalize].head())
```

```
Normalized Data (First 5 rows):
     Income  Loan_Amount
0  0.474695     0.285323
1  0.637137     0.407753
2  0.297850     0.789369
3  0.824364     0.323415
4  0.042361     0.514898
```

3. Standardization (Z-score Scaling)

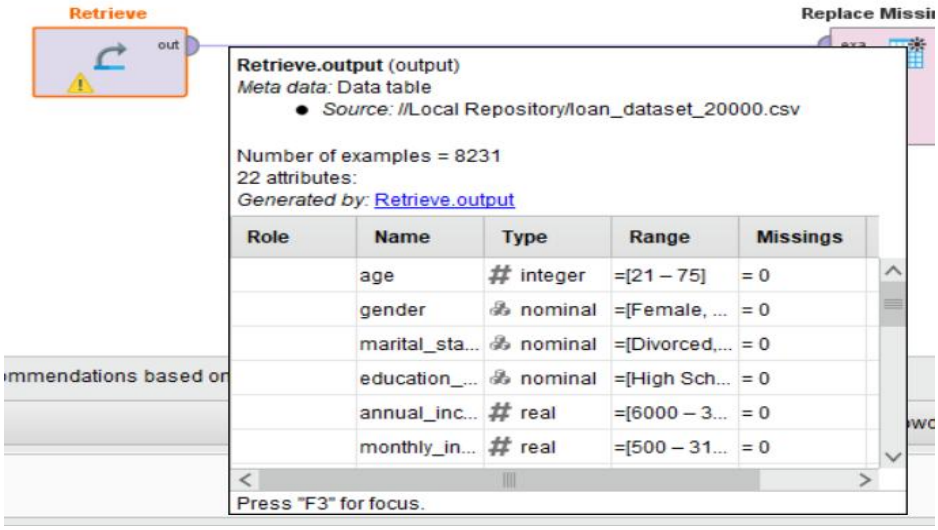
Standardization centers the data around a mean of 0 with a standard deviation of 1. It is less affected by outliers compared to normalization and is often used for algorithms like Logistic Regression or SVM.



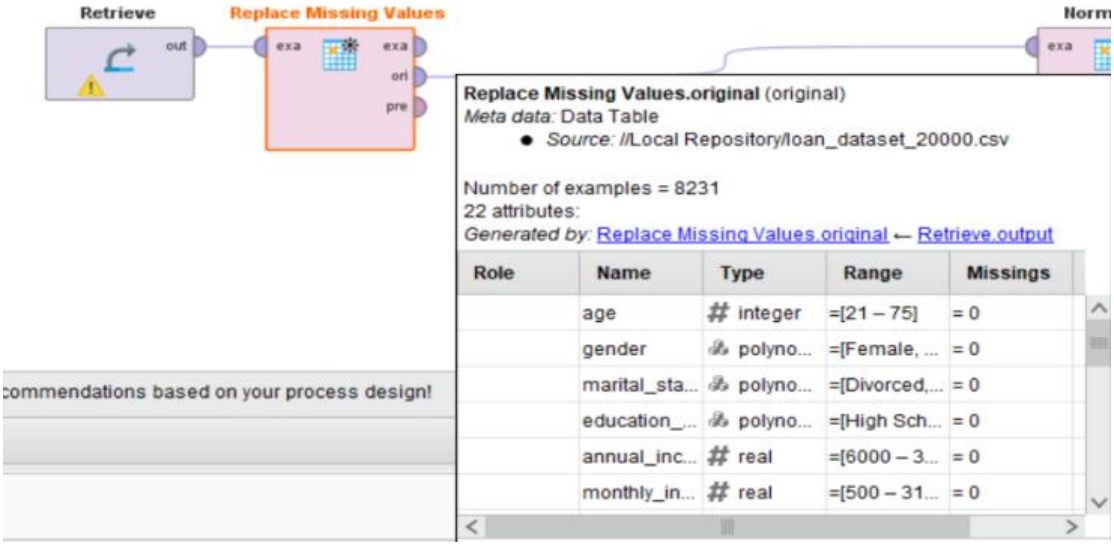
4. Discretization (Binning)

Discretization converts continuous variables into discrete categories (bins). This can help simplify the data or handle non-linear relationships.

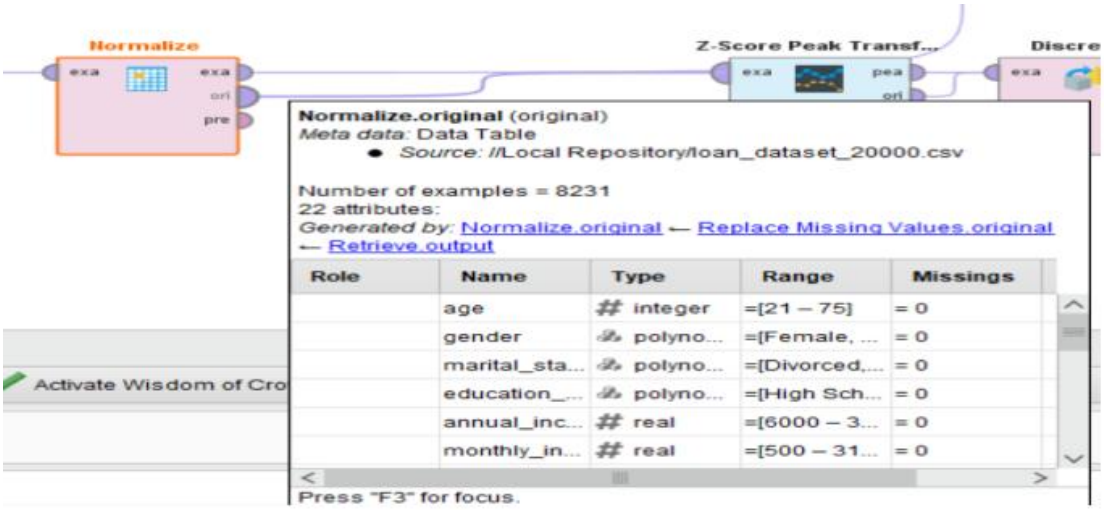
1. Load data



2. Handling missing values



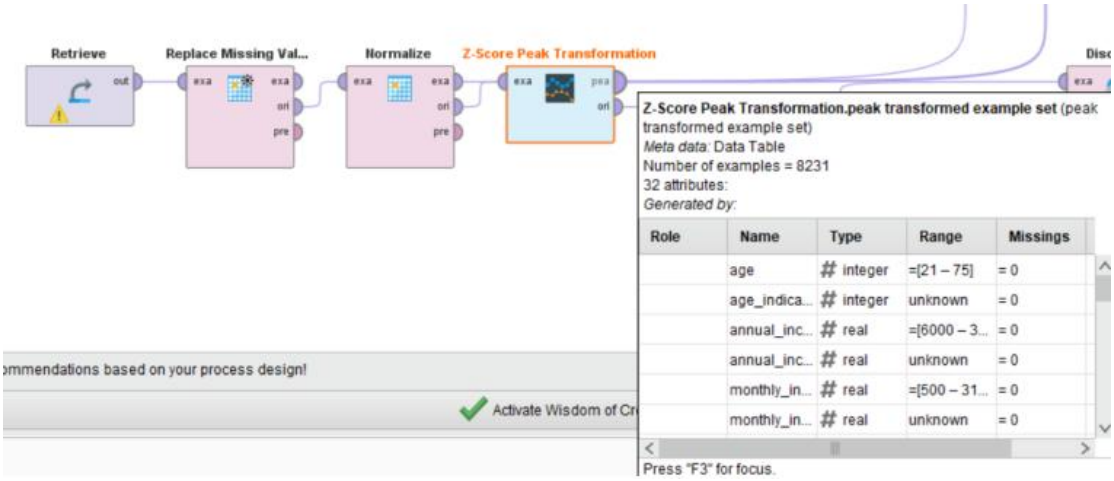
3. Normalization

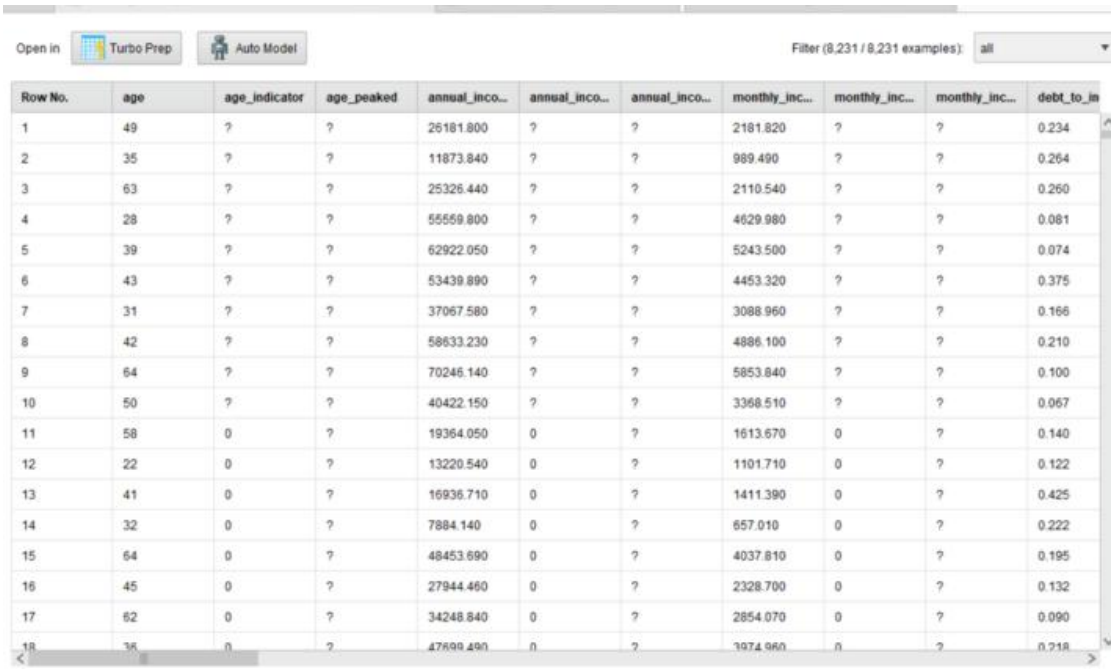


Open in Turbo Prep Auto Model Filter (8,231 / 8,231 examples): all

Row No.	age	annual_inco...	monthly_inc...	debt_to_inc...	credit_score	loan_amount	interest_rate	loan_term	installment	num_of_o
1	0.068	-0.595	-0.595	0.546	1.438	-1.350	-1.178	1.488	-1.391	-1.337
2	-0.817	-1.085	-1.085	0.833	1.870	-0.032	-1.803	-0.672	0.043	0.870
3	0.952	-0.624	-0.624	0.795	-0.230	0.156	1.126	1.488	-0.222	-1.778
4	-1.259	0.409	0.409	-0.920	1.366	-0.290	0.123	1.488	-0.617	-0.895
5	-0.564	0.661	0.661	-0.987	0.647	-1.715	-1.007	-0.672	-1.611	0.429
6	-0.311	0.337	0.337	1.896	1.683	-0.058	-0.251	-0.672	0.114	-0.454
7	-1.069	-0.223	-0.223	-0.106	0.618	1.123	1.110	-0.672	1.490	-0.895
8	-0.374	0.515	0.515	0.316	-1.453	0.105	1.955	1.488	-0.196	-0.454
9	1.015	0.912	0.912	-0.738	-3.438	-0.741	0.769	-0.672	-0.557	-0.454
10	0.131	-0.108	-0.108	-1.054	1.064	0.490	-0.259	-0.672	0.684	0.429
11	0.636	-0.828	-0.828	-0.355	-0.863	0.340	0.574	-0.672	0.593	1.312
12	-1.638	-1.038	-1.038	-0.527	0.431	-0.643	0.509	-0.672	-0.463	0.429
13	-0.438	-0.911	-0.911	2.375	0.374	0.398	0.444	-0.672	0.645	-0.895
14	-1.006	-1.221	-1.221	0.431	0.187	-1.100	-0.068	-0.672	-0.966	1.753
15	1.015	0.166	0.166	0.172	-0.992	-1.157	-0.775	1.488	-1.254	-1.778
16	-0.185	-0.535	-0.535	-0.431	-1.812	-1.043	1.049	-0.672	-0.875	-0.895
17	0.889	-0.319	-0.319	-0.833	0.086	-0.529	-0.304	-0.672	-0.378	0.870
18	-0.753	0.141	0.141	0.392	1.827	0.928	-2.067	-0.672	0.962	-0.012

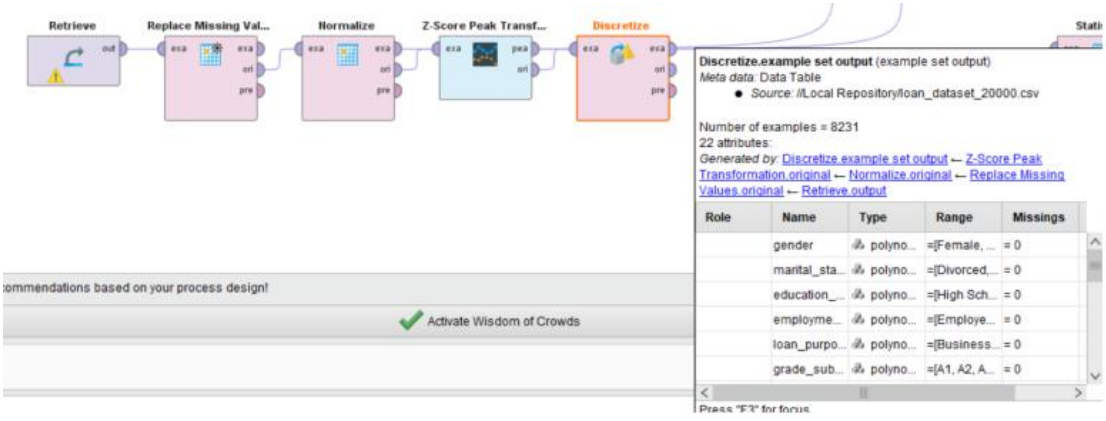
4. **Z-Score Peak Transformation**



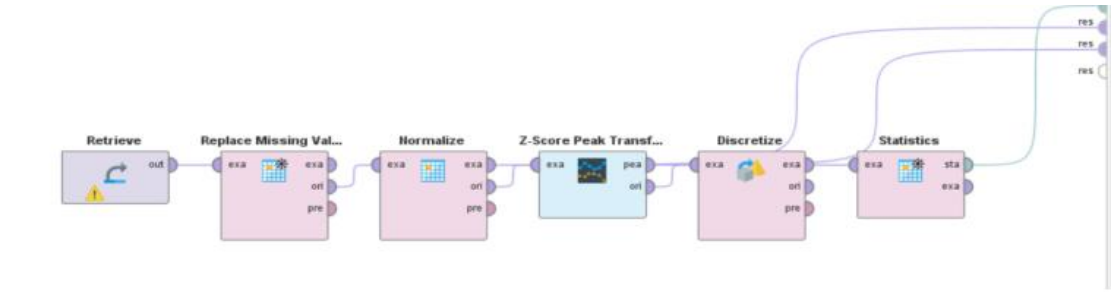


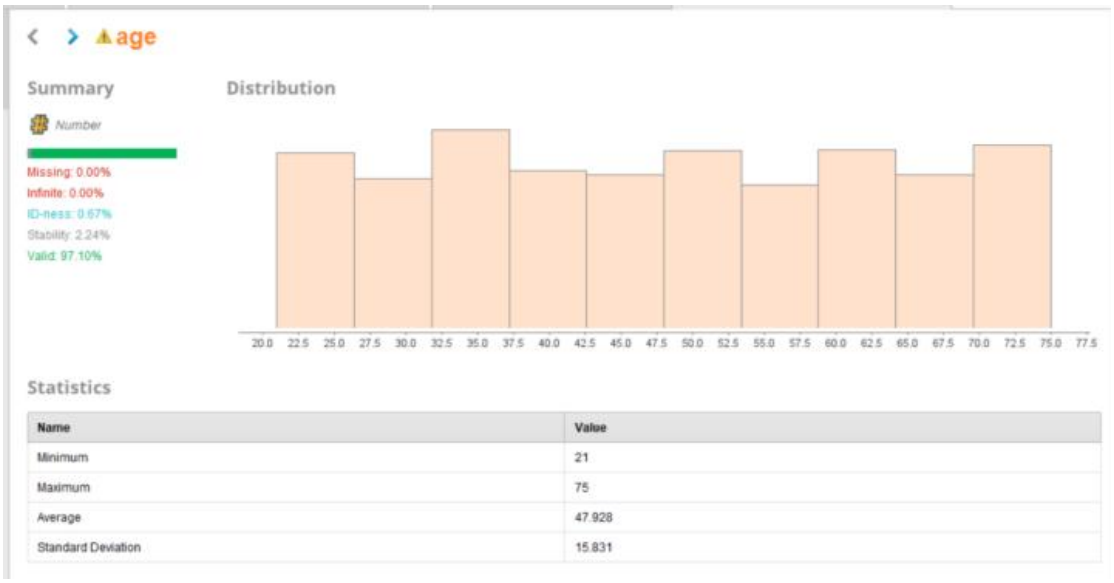
Row No.	age	age_indicator	age_peaked	annual_inco...	annual_inco...	annual_inco...	monthly_inc...	monthly_inc...	monthly_inc...	debt_to_in
1	49	?	?	26181.800	?	?	2181.820	?	?	0.234
2	35	?	?	11873.840	?	?	989.490	?	?	0.264
3	63	?	?	25326.440	?	?	2110.540	?	?	0.260
4	28	?	?	55559.800	?	?	4629.980	?	?	0.081
5	39	?	?	62922.050	?	?	5243.500	?	?	0.074
6	43	?	?	53439.890	?	?	4453.320	?	?	0.375
7	31	?	?	37067.580	?	?	3088.960	?	?	0.166
8	42	?	?	58633.230	?	?	4886.100	?	?	0.210
9	64	?	?	70246.140	?	?	5853.840	?	?	0.100
10	50	?	?	40422.150	?	?	3368.510	?	?	0.067
11	58	0	?	19364.050	0	?	1613.670	0	?	0.140
12	22	0	?	13220.540	0	?	1101.710	0	?	0.122
13	41	0	?	16936.710	0	?	1411.390	0	?	0.425
14	32	0	?	7884.140	0	?	657.010	0	?	0.222
15	64	0	?	48453.690	0	?	4037.810	0	?	0.195
16	45	0	?	27944.460	0	?	2328.700	0	?	0.132
17	62	0	?	34248.840	0	?	2854.070	0	?	0.090
18	35	0	?	47699.490	0	?	3974.960	0	?	0.218

5. Discretization



6, statistics





Statistics

Name	Value
Minimum	21
Maximum	75
Average	47.928
Standard Deviation	15.831

LAB-5

Market basket analysis , Support, confidence, lift calculation manually

Market Basket Analysis (MBA)

Definition:
Market Basket Analysis is a technique in Data Mining used to find relationships between items that customers buy together.

Itemset

A group of items
Example: {Milk, Bread}

Transaction

A single purchase record
Example: T1 = {Milk, Bread, Butter}

1. Support

Definition:
Support shows how frequently an itemset appears in the dataset.

Formula:

$$Support(A) = \frac{\text{Number of transactions containing A}}{\text{Total transactions}}$$

2. Confidence

Definition:
Confidence shows how likely item B is purchased when item A is purchased.

Formula:

$$Confidence(A \rightarrow B) = \frac{Support(A \cup B)}{Support(A)}$$

3. Lift

Definition:
Lift tells how strong the relationship is between A and B.

Formula:

$$Lift(A \rightarrow B) = \frac{Confidence(A \rightarrow B)}{Support(B)}$$

Interpretation:

- Lift = 1 → No relation
- Lift > 1 → Positive relation
- Lift < 1 → Negative relation

Load dataset : https://www.kaggle.com/datasets/janesmile/transaction-data

```
[1]: import pandas as pd
```

```
[2]: df = pd.read_csv('transaction_data.csv')
```

Preprocessing :

```
[3]: # 2. FIX: Remove rows where ItemDescription is missing
     df = df.dropna(subset=['ItemDescription'])

     # 3. Ensure all descriptions are strings (just in case there are numeric codes)
     df['ItemDescription'] = df['ItemDescription'].astype(str)
```

```
[4]: # 2. Preprocessing: Create a list of items per transaction
     # We group by TransactionId and collect ItemDescription into a list
     basket = df.groupby('TransactionId')['ItemDescription'].apply(list).reset_index()
```

Define items :

```
[5]: # 3. Define the Items for Analysis
     # Let's say we want to calculate the relationship between Item A and Item B
     item_A = 'WHITE HANGING HEART T-LIGHT HOLDER'  # Replace with actual item names from your data
     item_B = 'KNITTED UNION FLAG HOT WATER BOTTLE'
```

Manual calculations and define transactions :

```
[6]: # 4. Manual Calculations
     total_transactions = len(basket)
```

```
[7]: # Transactions containing A, B, and both
     count_A = basket['ItemDescription'].apply(lambda x: item_A in x).sum()
     count_B = basket['ItemDescription'].apply(lambda x: item_B in x).sum()
     count_both = basket['ItemDescription'].apply(lambda x: item_A in x and item_B in x).sum()
```

```
[8]: # Calculations
     support_A = count_A / total_transactions
     support_B = count_B / total_transactions
     support_both = count_both / total_transactions

     confidence = count_both / count_A if count_A > 0 else 0
     lift = confidence / support_B if support_B > 0 else 0
```

Total transactions :

```
[9]: # 5. Output Results
     print(f"Total Transactions: {total_transactions}")

     Total Transactions: 24446
```

Support :

```
[10]: print(f"Support({item_A}): {support_A:.4f}")

      Support(WHITE HANGING HEART T-LIGHT HOLDER): 0.0942

[11]: print(f"Support({item_B}): {support_B:.4f}")

      Support(KNITTED UNION FLAG HOT WATER BOTTLE): 0.0189
```

Confidence :

```
[12]: print(f"Confidence({item_A} -> {item_B}): {confidence:.4f}")

      Confidence(WHITE HANGING HEART T-LIGHT HOLDER -> KNITTED UNION FLAG HOT WATER BOTTLE): 0.0665
```

Lift :

```
[13]: print(f"Lift({item_A} -> {item_B}): {lift:.4f}")

      Lift(WHITE HANGING HEART T-LIGHT HOLDER -> KNITTED UNION FLAG HOT WATER BOTTLE): 3.5092
```

Validation :

```
[14]: from mlxtend.preprocessing import TransactionEncoder
      from mlxtend.frequent_patterns import apriori, association_rules

[15]: # Create Basket
      basket_list = df.groupby('TransactionId')['ItemDescription'].apply(list).tolist()

      # One-hot encoding
      te = TransactionEncoder()
      te_ary = te.fit(basket_list).transform(basket_list)
      df_encoded = pd.DataFrame(te_ary, columns=te.columns_)
```

LAB-6

Apriori algorithm using Python, Frequent itemset generation

Apriori Algorithm

Definition:
The Apriori algorithm is a fundamental algorithm in Data Mining used to find **frequent itemsets** and generate **association rules**.

1. Itemset

A set of items
Example: {Milk, Bread}

2. Frequent Itemset

An itemset whose **support ≥ minimum support threshold**

Apriori Property

If an itemset is frequent → all its subsets are also frequent
If an itemset is NOT frequent → all its supersets are NOT frequent

How Apriori Works

1. Generate candidate itemsets
2. Calculate support
3. Remove those below threshold
4. Repeat until no more itemsets

Load dataset : DATA MINING\market_basket.csv

```
[1]: import pandas as pd
     from mlxtend.preprocessing import TransactionEncoder
     from mlxtend.frequent_patterns import apriori, association_rules

[2]: df = pd.read_csv("market_basket.csv")
```

Convert into transactions :

```
[3]: transactions = df.groupby('TransactionID')['Item'].apply(list).tolist()

[4]: print("Transactions:")
     print(transactions)

     Transactions:
     [['Milk', 'Bread', 'Butter'], ['Milk', 'Bread'], ['Milk', 'Butter'], ['Bread', 'Butter'], ['Milk', 'Bread', 'Butter']]
```

One hot encoding :

```
[5]: te = TransactionEncoder()
     te_array = te.fit(transactions).transform(transactions)
     df_encoded = pd.DataFrame(te_array, columns=te.columns_)

     print("\nEncoded Data:")
     print(df_encoded)
```

```
Encoded Data:
   Bread  Butter   Milk
0   True    True   True
1   True   False   True
2  False    True   True
3   True    True  False
4   True    True   True
```

Generate frequent itemset :

```
[6]: frequent_itemsets = apriori(df_encoded, min_support=0.4, use_colnames=True)

     print("\nFrequent Itemsets:")
     print(frequent_itemsets)
```

```
Frequent Itemsets:
   support              itemsets
0      0.8               (Bread)
1      0.8              (Butter)
2      0.8                (Milk)
3      0.6        (Bread, Butter)
4      0.6          (Bread, Milk)
5      0.6         (Butter, Milk)
6      0.4  (Bread, Butter, Milk)
```

Generate association rules:

```
[8]: rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.6)

     print("\nAssociation Rules:")
     display(rules)
```

Association Rules:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczyn
0	(Bread)	(Butter)	0.8	0.8	0.6	0.750000	0.937500	1.0	-0.04	0.8	-0.250000	0.6	-0.250000	0.750
1	(Butter)	(Bread)	0.8	0.8	0.6	0.750000	0.937500	1.0	-0.04	0.8	-0.250000	0.6	-0.250000	0.750
2	(Bread)	(Milk)	0.8	0.8	0.6	0.750000	0.937500	1.0	-0.04	0.8	-0.250000	0.6	-0.250000	0.750
3	(Milk)	(Bread)	0.8	0.8	0.6	0.750000	0.937500	1.0	-0.04	0.8	-0.250000	0.6	-0.250000	0.750
4	(Butter)	(Milk)	0.8	0.8	0.6	0.750000	0.937500	1.0	-0.04	0.8	-0.250000	0.6	-0.250000	0.750
5	(Milk)	(Butter)	0.8	0.8	0.6	0.750000	0.937500	1.0	-0.04	0.8	-0.250000	0.6	-0.250000	0.750
6	(Bread, Butter)	(Milk)	0.6	0.8	0.4	0.666667	0.833333	1.0	-0.08	0.6	-0.333333	0.4	-0.666667	0.583
7	(Bread, Milk)	(Butter)	0.6	0.8	0.4	0.666667	0.833333	1.0	-0.08	0.6	-0.333333	0.4	-0.666667	0.583
8	(Butter, Milk)	(Bread)	0.6	0.8	0.4	0.666667	0.833333	1.0	-0.08	0.6	-0.333333	0.4	-0.666667	0.583

LAB-7

Construct FP-Tree, compare with Apriori

FP-Tree (FP-Growth)

The FP-Growth algorithm is an advanced method in Data Mining used to generate frequent itemsets **without candidate generation** (unlike Apriori). Instead of generating combinations, it builds a **compact tree structure called FP-Tree (Frequent Pattern Tree)**.

Mining Frequent Patterns from FP-Tree

- Start from **least frequent item**
- Build **conditional pattern base**
- Construct **conditional FP-tree**
- Extract frequent itemsets

No need to generate all combinations

Apriori vs FP-Growth

Feature	Apriori	FP-Growth
Algorithm	Apriori algorithm	FP-Growth algorithm
Candidate Generation	Yes	No
Database Scans	Multiple	No
Speed	Slow	Fast
Memory	High	Efficient
Best For	Small datasets	Large datasets

FP-growth :

```
[1]: import pandas as pd
     from mlxtend.preprocessing import TransactionEncoder
     from mlxtend.frequent_patterns import fpgrowth, association_rules

[2]: df = pd.read_csv("retail_fp_dataset.csv")

[3]: transactions = df.groupby('TransactionID')['Item'].apply(list).tolist()

     # Encoding
     te = TransactionEncoder()
     te_array = te.fit(transactions).transform(transactions)
     df_encoded = pd.DataFrame(te_array, columns=te.columns_)
```

```
[5]: # FP-Growth
fp_itemsets = fpgrowth(df_encoded, min_support=0.3, use_colnames=True)

print("FP-Growth Frequent Itemsets:")
display(fp_itemsets)
```

FP-Growth Frequent Itemsets:

	support	itemsets
0	0.7	(Milk)
1	0.7	(Bread)
2	0.6	(Butter)
3	0.4	(Eggs)
4	0.4	(Bread, Milk)
5	0.4	(Bread, Butter)
6	0.4	(Milk, Butter)
7	0.3	(Bread, Eggs)

```
[6]: # Association Rules
fp_rules = association_rules(fp_itemsets, metric="confidence", min_threshold=0.6)

print("\nFP-Growth Rules:")
display(fp_rules)
```

FP-Growth Rules:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulcz
0	(Butter)	(Bread)	0.6	0.7	0.4	0.666667	0.952381	1.0	-0.02	0.9	-0.111111	0.444444	-0.111111	0.61
1	(Butter)	(Milk)	0.6	0.7	0.4	0.666667	0.952381	1.0	-0.02	0.9	-0.111111	0.444444	-0.111111	0.61
2	(Eggs)	(Bread)	0.4	0.7	0.3	0.750000	1.071429	1.0	0.02	1.2	0.111111	0.375000	0.166667	0.5

Apriori :

```
[7]: from mlxtend.frequent_patterns import apriori
```

```
[8]: # Apriori
ap_itemsets = apriori(df_encoded, min_support=0.3, use_colnames=True)

print("Apriori Frequent Itemsets:")
display(ap_itemsets)
```

Apriori Frequent Itemsets:

	support	itemsets
0	0.7	(Bread)
1	0.6	(Butter)
2	0.4	(Eggs)
3	0.7	(Milk)
4	0.4	(Bread, Butter)
5	0.3	(Bread, Eggs)
6	0.4	(Bread, Milk)
7	0.4	(Milk, Butter)

```
[9]: # Rules
ap_rules = association_rules(ap_itemsets, metric="confidence", min_threshold=0.6)

print("\nApriori Rules:")
display(ap_rules)
```

Apriori Rules:

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulcz
0	(Butter)	(Bread)	0.6	0.7	0.4	0.666667	0.952381	1.0	-0.02	0.9	-0.111111	0.444444	-0.111111	0.61
1	(Eggs)	(Bread)	0.4	0.7	0.3	0.750000	1.071429	1.0	0.02	1.2	0.111111	0.375000	0.166667	0.58
2	(Butter)	(Milk)	0.6	0.7	0.4	0.666667	0.952381	1.0	-0.02	0.9	-0.111111	0.444444	-0.111111	0.61

Comparison code :

```
[10]: import time

# Apriori timing
start = time.time()
apriori(df_encoded, min_support=0.3, use_colnames=True)
ap_time = time.time() - start

# FP-Growth timing
start = time.time()
fpgrowth(df_encoded, min_support=0.3, use_colnames=True)
fp_time = time.time() - start

print("Apriori Time:", ap_time)
print("FP-Growth Time:", fp_time)
```

```
Apriori Time: 0.006995677947998047
FP-Growth Time: 0.0059969425201416016
```

LAB-8

Lists, dictionaries, class creation, preprocessing methods

1. Python Syntax

Python syntax refers to the set of rules that defines how Python programs are written and interpreted. It is designed to be simple and readable, making it beginner-friendly. Python uses **indentation** instead of brackets to define code blocks such as loops, functions, and conditions.

```
[1]: # Print statement
     print("Hello Data Mining!")

     Hello Data Mining!
```

Variables are created by assigning values directly without declaring their type, as Python is a dynamically typed language. Basic syntax includes print statements, input handling, comments, and control structures like if-else and loops. This simplicity makes Python widely used in data science, machine learning, and data mining applications.

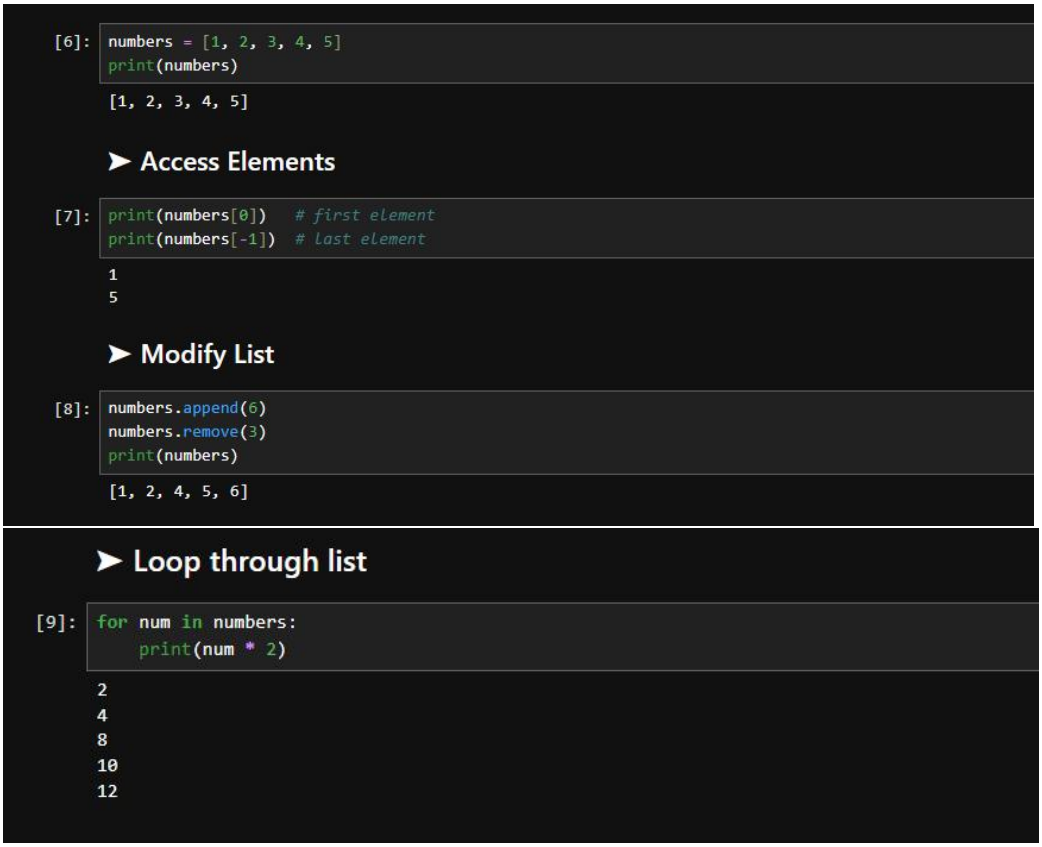
```
[3]: # Variables
     x = 10
     y = 20
     z = x + y
     print(z)

     30
```

```
[5]: # Data types
     a = 10        # int
     b = 3.14      # float
     c = "AI"      # string
     d = True      # boolean

     print(type(a))
     print(type(b))
     print(type(c))
     print(type(d))

     <class 'int'>
     <class 'float'>
     <class 'str'>
     <class 'bool'>
```

2. Lists in Python

A list in Python is an ordered, **mutable** (changeable) collection that can store multiple values in a single variable. Lists can contain elements of different data types such as integers, strings, or even other lists. Each element in a list is indexed starting from zero, which allows easy **access and**

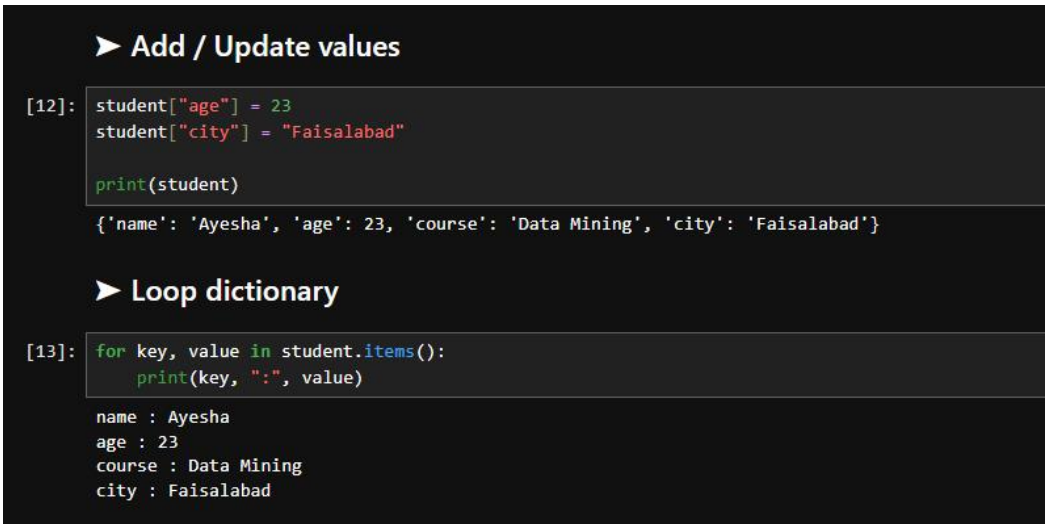
modification. Lists are very useful in data mining because they help store and manipulate datasets, transaction records, and feature values. Common operations on lists include adding elements, removing elements, slicing, and iterating through items using loops.



3. Dictionaries in Python

A dictionary in Python is an **unordered collection** of data stored in **key-value pairs.** Each key is unique and is used to access its corresponding value. Dictionaries are highly efficient for searching and retrieving data because they use a hashing mechanism. They are widely used in data mining to represent structured data such as records, datasets, and feature mappings.

For example, a student record can be stored using keys like name, age, and course. Dictionaries support operations such as adding, updating, and deleting key-value pairs.

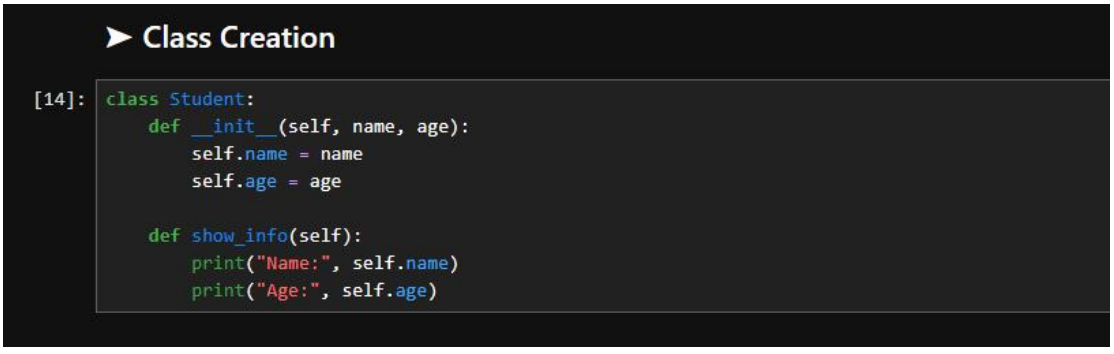


4. OOP Basics

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects and classes. A class is a blueprint that defines properties (attributes) and behaviors (methods), while an object is an instance of a class. OOP helps in organizing code in a modular and reusable way. It includes key concepts such as encapsulation, inheritance, polymorphism, and abstraction. In data mining and machine learning, OOP is used to structure preprocessing pipelines, model building, and reusable components for better code management and scalability.

Class Creation in Python

In Python, a class is created using the class keyword and is used to define a structure that combines data and functions. The constructor method __init__ is automatically called when an object is created and is used to initialize variables. Methods inside a class define the behavior of objects. Class creation allows developers to group related functions and data together, making programs more organized and reusable. In data mining, classes are often used to build preprocessing tools, custom algorithms, and machine learning pipelines.



➤ Create Object

```
[15]: s1 = Student("Ayesha", 22)
      s1.show_info()
```

```
Name: Ayesha
Age: 22
```

➤ Simple Data Mining Style Class Example

```
[16]: class DataPreprocessor:

          def __init__(self, data):
              self.data = data

          def remove_missing(self):
              self.data = [x for x in self.data if x is not None]
              return self.data

          def normalize(self):
              max_val = max(self.data)
              self.data = [x / max_val for x in self.data]
              return self.data
```

```
[17]: data = [10, 20, None, 40, 50]

      processor = DataPreprocessor(data)

      print(processor.remove_missing())
      print(processor.normalize())
```

```
[10, 20, 40, 50]
[0.2, 0.4, 0.8, 1.0]
```

6. Preprocessing Methods in Data Mining

Data preprocessing is an essential step in data mining that involves cleaning and transforming raw data into a suitable format for analysis. Real-world data often contains missing values, noise, inconsistencies, and outliers, which must be handled before applying machine learning algorithms. Common preprocessing techniques include **handling missing values using mean, median, or mode; encoding categorical variables into numerical form; scaling features using normalization or standardization; and removing duplicates or outliers**. Preprocessing improves data quality, increases model accuracy, and ensures better performance of data mining algorithms.

0. Import Libraries

```
[18]: import pandas as pd
      import numpy as np
```

1. Load Dataset

```
[19]: df = pd.read_csv("preprocessing_dataset.csv")
      df.head()
```

[19]:

	Age	Salary	Gender	City	Target
0	56	79150	Female	Faisalabad	0
1	46	85725	Female	Lahore	0
2	32	104654	Female	Karachi	0
3	25	55773	Female	Faisalabad	0
4	38	87435	Female	Islamabad	1

2. Check Missing Values

```
[20]: df.isnull().sum()
```

```
[20]: Age       0
      Salary    0
      Gender    0
      City      0
      Target    0
      dtype: int64
```

3. Handle Missing Values

➤ Fill with Mean (numeric)

```
[22]: df["Age"] = df["Age"].fillna(df["Age"].mean())
```

➤ Fill with Mode (categorical)

```
[23]: df["Gender"] = df["Gender"].fillna(df["Gender"].mode())
```

➤ Drop missing values

```
[24]: df.dropna(inplace=True)
```

4. Remove Duplicates

```
[25]: df.drop_duplicates(inplace=True)
```

5. Encoding Categorical Data

Label Encoding

[26]:
```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df["Gender"] = le.fit_transform(df["Gender"])
```

➤ One Hot Encoding

[27]:
```
df = pd.get_dummies(df, columns=["City"])
```

6. Feature Scaling

➤ Min-Max Normalization (0 to 1)

[28]:
```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

df[["Salary"]] = scaler.fit_transform(df[["Salary"]])
```

➤ Standardization (Z-score)

[29]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

df[["Salary"]] = scaler.fit_transform(df[["Salary"]])
```

7. Outlier Detection & Removal (IQR Method)

[30]:
```
Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)

IQR = Q3 - Q1

df = df[(df["Salary"] >= Q1 - 1.5 * IQR) &
        (df["Salary"] <= Q3 + 1.5 * IQR)]
```

8. Feature Selection

[31]:
```
df = df[["Age", "Salary", "Gender"]]
```

9. Train-Test Split

[34]:
```
df.columns = df.columns.str.strip()

df["Target"] = df["Target"] if "Target" in df.columns else None
```

[36]:
```
from sklearn.model_selection import train_test_split

X = df.drop("Target", axis=1)
y = df["Target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

10. Data Type Conversion

[37]:
```
df["Age"] = df["Age"].astype(int)
df["Salary"] = df["Salary"].astype(float)
```

11. Feature Scaling on Multiple Columns

[38]:
```
cols = ["Age", "Salary"]

scaler = MinMaxScaler()
df[cols] = scaler.fit_transform(df[cols])
```

12. Binning (Discretization)

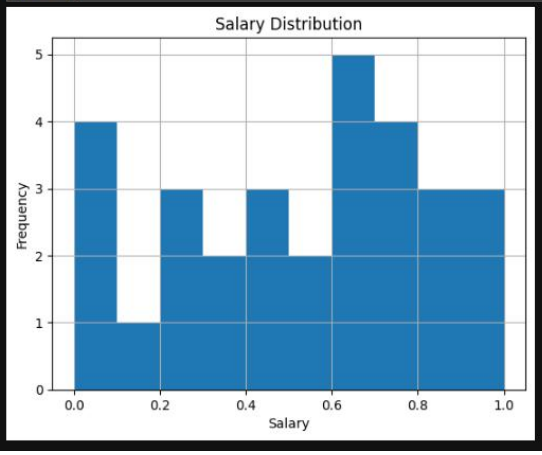
[40]:
```
df["Age_Group"] = pd.cut(df["Age"], bins=[0, 18, 35, 60, 100],
                         labels=["Teen", "Young", "Adult", "Senior"])
```

13. DATA VISUALIZATION

[41]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

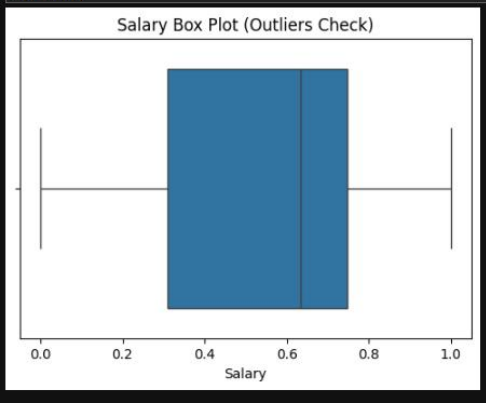
1. Histogram (Distribution of Numeric Data)

```
[42]: df["Salary"].hist()
      plt.title("Salary Distribution")
      plt.xlabel("Salary")
      plt.ylabel("Frequency")
      plt.show()
```



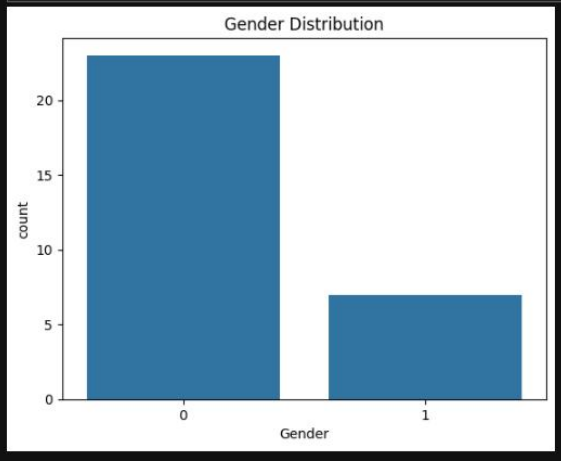
2. Box Plot (Outliers Detection)

```
[43]: plt.figure(figsize=(6,4))
      sns.boxplot(x=df["Salary"])
      plt.title("Salary Box Plot (Outliers Check)")
      plt.show()
```



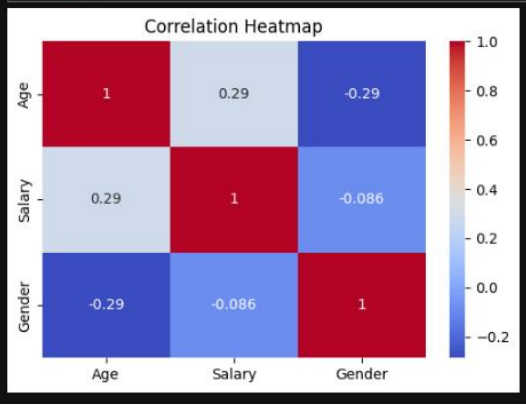
3. Count Plot (Categorical Data)

```
[44]: sns.countplot(x=df["Gender"])
      plt.title("Gender Distribution")
      plt.show()
```



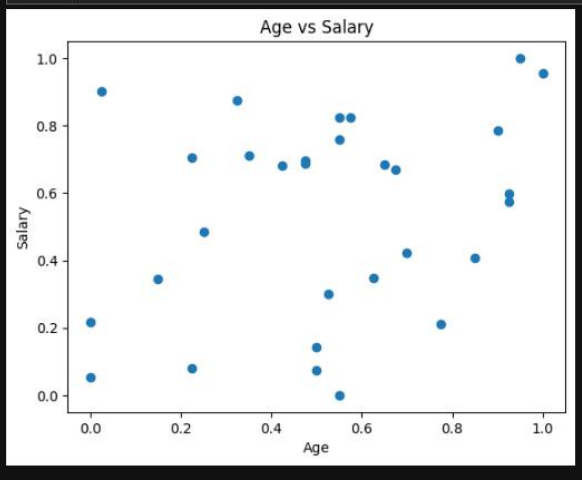
4. Correlation Heatmap

```
[45]: plt.figure(figsize=(6,4))
      sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
      plt.title("Correlation Heatmap")
      plt.show()
```



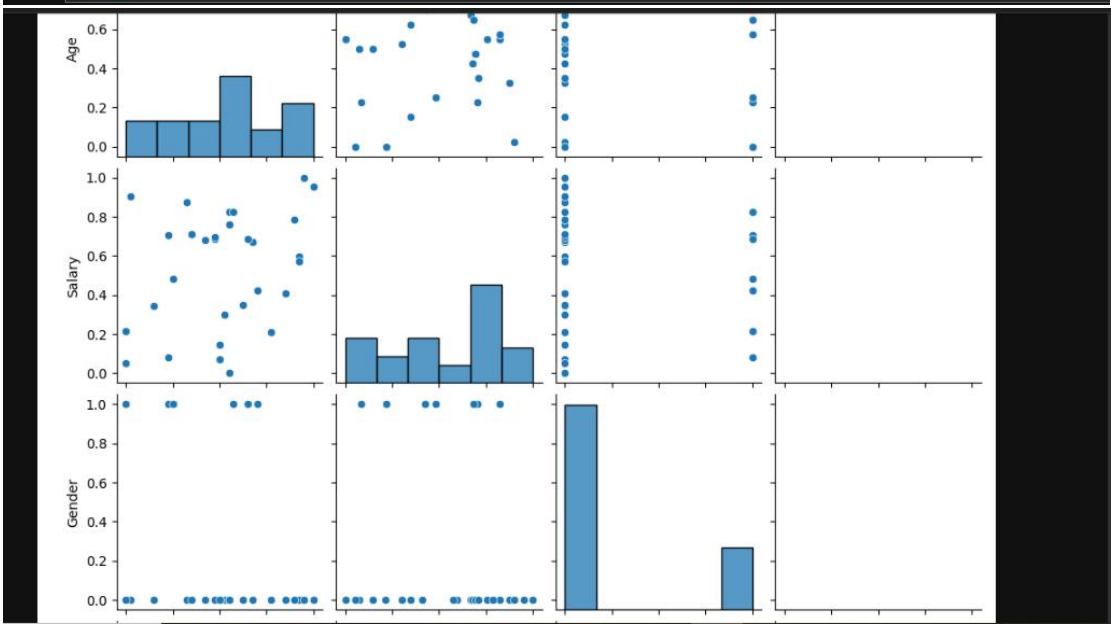
5. Scatter Plot (Relationship between variables)

```
[46]: plt.scatter(df["Age"], df["Salary"])
      plt.title("Age vs Salary")
      plt.xlabel("Age")
      plt.ylabel("Salary")
      plt.show()
```



6. Pairplot (Full Overview of Dataset)

```
[47]: sns.pairplot(df)
      plt.show()
```



LAB-9

Decision Tree Classifier for Customer Purchase Prediction

Lab Title

Building a Decision Tree Classifier for Customer Purchase Prediction

Theory

Decision Tree Classifier

A **Decision Tree** is a supervised machine learning algorithm used for **classification** and **regression** tasks. It predicts the target variable by learning simple decision rules inferred from the training data.

The model creates a tree-like structure consisting of:

1. Root Node

Represents the most important feature used for splitting the data.

2. Internal Nodes

Represent conditions or tests on features.

3. Leaf Nodes

Represent the final prediction or class label.

How Decision Trees Work

1. Start with the complete dataset.
2. Select the best feature for splitting using criteria such as:

- Gini Index or Entropy (Information Gain)

3. Divide the dataset into subsets.
4. Repeat the process recursively until:

- Maximum depth is reached, or
- Nodes become pure.

Advantages

- Easy to understand and interpret.
- Requires little data preprocessing.
- Handles both numerical and categorical data.
- Visualization is straightforward.

Disadvantages

- Can easily overfit the data.
- Sensitive to small changes in data.
- May create biased trees if classes are imbalanced.

Dataset

Use the **Customer Personality Analysis Dataset** available on Kaggle.

Dataset Link:

Customer Personality Analysis Dataset

This dataset contains customer demographics and purchasing behavior useful for marketing analysis.

Jupyter Notebook Code

Step 1: Import Libraries

```
[1]: import pandas as pd
     import numpy as np

     from sklearn.model_selection import train_test_split
     from sklearn.tree import DecisionTreeClassifier
     from sklearn.tree import plot_tree

     from sklearn.metrics import accuracy_score
     from sklearn.metrics import precision_score
     from sklearn.metrics import recall_score
     from sklearn.metrics import f1_score
     from sklearn.metrics import classification_report
     from sklearn.metrics import confusion_matrix

     import matplotlib.pyplot as plt
```

Step 2: Load Dataset

```
[2]: df = pd.read_csv("marketing_campaign.csv", sep='\t')

     # Display first 5 rows
     df.head()
```

[2]:

	ID	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	...	NumWebVisitsMonth	AcceptedCmp3	AcceptedCr
0	5524	1957	Graduation	Single	58138.0	0	0	04-09-2012	58	635	...	7	0	
1	2174	1954	Graduation	Single	46344.0	1	1	08-03-2014	38	11	...	5	0	
2	4141	1965	Graduation	Together	71613.0	0	0	21-08-2013	26	426	...	4	0	
3	6182	1984	Graduation	Together	26646.0	1	0	10-02-2014	26	11	...	6	0	
4	5324	1981	PhD	Married	58293.0	1	0	19-01-2014	94	173	...	5	0	

5 rows × 29 columns

Step 3: Check Dataset Information

```
[3]: print(df.info())

     print("\nMissing Values:")
     print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 29 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   ID                   2240 non-null   int64
 1   Year_Birth           2240 non-null   int64
 2   Education            2240 non-null   object
 3   Marital_Status       2240 non-null   object
 4   Income               2216 non-null   float64
 5   Kidhome              2240 non-null   int64
 6   Teenhome             2240 non-null   int64
 7   Dt_Customer          2240 non-null   object
 8   Recency              2240 non-null   int64
 9   MntWines             2240 non-null   int64
 10  MntFruits            2240 non-null   int64
 11  MntMeatProducts      2240 non-null   int64
 12  MntFishProducts      2240 non-null   int64
 13  MntSweetProducts     2240 non-null   int64
 14  MntGoldProds         2240 non-null   int64
 15  NumDealsPurchases    2240 non-null   int64
```

Step 4: Data Preprocessing

Remove unnecessary columns

```
[4]: df.drop(['ID', 'Dt_Customer', 'Z_CostContact', 'Z_Revenue'],
             axis=1,
             inplace=True)
```

Handle Missing Values

```
[5]: df['Income'].fillna(df['Income'].median(), inplace=True)
```

Convert Categorical Variables into Numeric

```
[6]: df = pd.get_dummies(df,
                        columns=['Education', 'Marital_Status'],
                        drop_first=True)
```

Step 5: Define Features and Target Variable

```
[7]: X = df.drop('Response', axis=1)

     y = df['Response']
```

Step 6: Split Dataset into Training and Testing Sets

```
[8]: X_train, X_test, y_train, y_test = train_test_split(
         X,
         y,
         test_size=0.2,
         random_state=42
     )
```

Step 7: Build Decision Tree Classifier

```
[9]: dt_model = DecisionTreeClassifier(
         criterion='gini',
         max_depth=4,
         random_state=42
     )

     dt_model.fit(X_train, y_train)
```

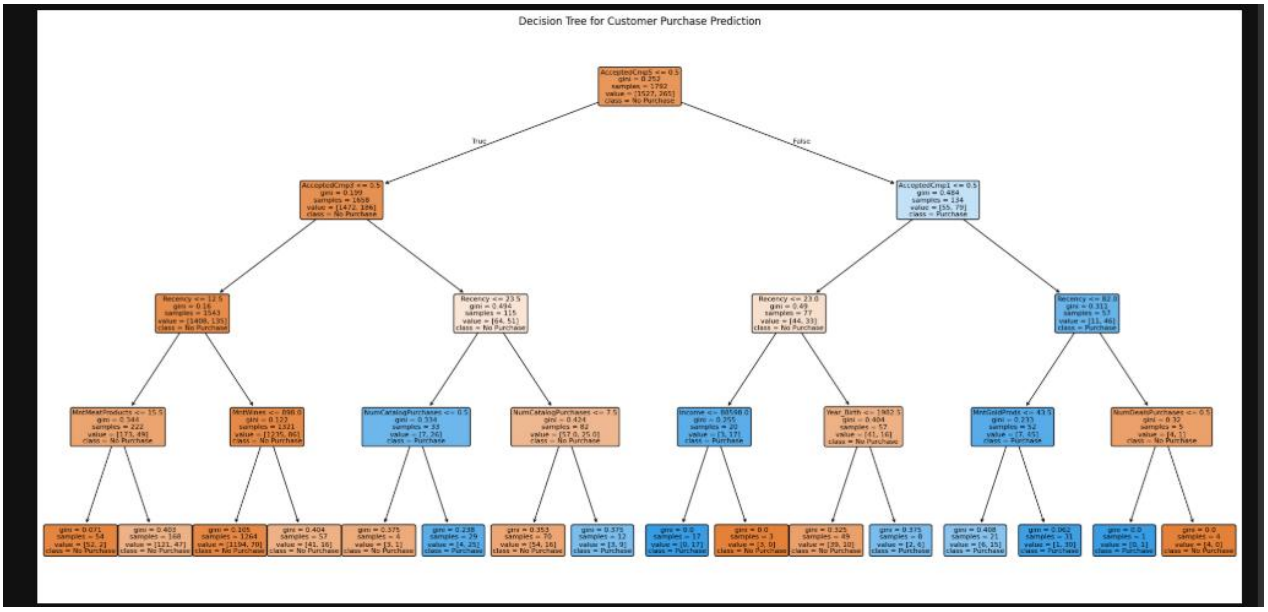
```
[9]: ▾ DecisionTreeClassifier
     ▶ Parameters
```

Step 8: Make Predictions

```
[10]: y_pred = dt_model.predict(X_test)
```

Step 9: Visualize the Decision Tree

```
[11]: plt.figure(figsize=(25, 12))

      plot_tree(
          dt_model,
          feature_names=X.columns,
          class_names=['No Purchase', 'Purchase'],
          filled=True,
          rounded=True,
          fontsize=8
      )

      plt.title("Decision Tree for Customer Purchase Prediction")
      plt.show()
```



Step 10: Evaluate Model Performance

Accuracy

```
[12]: accuracy = accuracy_score(y_test, y_pred)

      print("Accuracy:", accuracy)

      Accuracy: 0.8526785714285714
```

Precision

```
[13]: precision = precision_score(y_test, y_pred)

      print("Precision:", precision)

      Precision: 0.5714285714285714
```

Recall

```
[14]: recall = recall_score(y_test, y_pred)

      print("Recall:", recall)

      Recall: 0.17391304347826086
```

F1-Score

```
[15]: f1 = f1_score(y_test, y_pred)

      print("F1-Score:", f1)

      F1-Score: 0.26666666666666666
```

Step 11: Classification Report

```
[16]: print("\nClassification Report:\n")

      print(classification_report(y_test, y_pred))
```

```
Classification Report:

              precision    recall  f1-score   support

           0       0.87      0.98      0.92       379
           1       0.57      0.17      0.27        69

    accuracy                           0.85       448
   macro avg       0.72      0.58      0.59       448
weighted avg       0.82      0.85      0.82       448
```

Step 12: Confusion Matrix

```
[17]: cm = confusion_matrix(y_test, y_pred)

      print(cm)
```

```
[[370   9]
 [ 57  12]]
```

LAB-10

Email Spam Detection using Naïve Bayes and K-Nearest Neighbors (KNN)

Lab Title

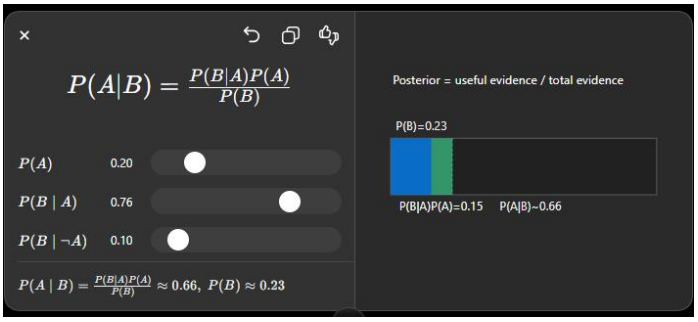
Implementation of Naïve Bayes and KNN Algorithms for Email Spam Detection and Comparison of Results

Theory

1. Naïve Bayes Classifier

Naïve Bayes is a **probabilistic supervised learning algorithm** based on **Bayes' Theorem**. It assumes that all features are independent of each other, which is why it is called "naïve."

Bayes' Theorem



Where:

- **P(A|B)** = Posterior probability
- **P(B|A)** = Likelihood
- **P(A)** = Prior probability
- **P(B)** = Evidence probability

2. K-Nearest Neighbors (KNN)

KNN is a **supervised machine learning algorithm** that classifies data based on the majority class among its **K nearest neighbors**.

Working of KNN

1. Choose the value of **K**.
2. Calculate the distance between the new data point and all training points.

3. Select the K nearest neighbors.
4. Assign the class with the majority vote.

Dataset

Use the **SMS Spam Collection Dataset** (commonly used for email/spam detection studies).

Dataset Link:

SMS Spam Collection Dataset (Kaggle)

Jupyter Notebook Code

Step 1: Import Libraries

```
[1]: import pandas as pd
     import numpy as np

     from sklearn.model_selection import train_test_split
     from sklearn.feature_extraction.text import TfidfVectorizer

     from sklearn.naive_bayes import MultinomialNB
     from sklearn.neighbors import KNeighborsClassifier

     from sklearn.metrics import accuracy_score
     from sklearn.metrics import precision_score
     from sklearn.metrics import recall_score
     from sklearn.metrics import f1_score
     from sklearn.metrics import classification_report
     from sklearn.metrics import confusion_matrix
```

Step 2: Load Dataset

```
[2]: df = pd.read_csv('spam.csv', encoding='latin-1')
```

Step 3: Display Dataset Information

```
[3]: print(df.head())

     print(df.info())
```

```
     v1                                                 v2 Unnamed: 2  \
0   ham  Go until jurong point, crazy.. Available only ...        NaN
1   ham                      Ok lar... Joking wif u oni...        NaN
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...        NaN
3   ham  U dun say so early hor... U c already then say...        NaN
4   ham  Nah I don't think he goes to usf, he lives aro...        NaN

  Unnamed: 3 Unnamed: 4
0        NaN        NaN
1        NaN        NaN
2        NaN        NaN
3        NaN        NaN
4        NaN        NaN
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   v1          5572 non-null   object
 1   v2          5572 non-null   object
 2   Unnamed: 2  50 non-null     object
```

Step 4: Data Preprocessing

```
[4]: df = df[['v1', 'v2']]

     df.columns = ['label', 'message']

     print(df.head())
```

```
  label                                            message
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...
```

Step 5: Convert Labels into Numeric Form

```
[5]: df['label'] = df['label'].map({
         'ham': 0,
         'spam': 1
     })
```

Step 6: Define Features and Target Variable

```
[6]: X = df['message']

     y = df['label']
```

Step 7: Convert Text into Numerical Features using TF-IDF

```
[7]: tfidf = TfidfVectorizer(stop_words='english')

     X_features = tfidf.fit_transform(X)
```

Step 8: Split Dataset

```
[8]: X_train, X_test, y_train, y_test = train_test_split(
         X_features,
         y,
         test_size=0.2,
         random_state=42
     )
```

Part A: Naïve Bayes Implementation

Step 9: Train Naïve Bayes Model

```
[9]: nb_model = MultinomialNB()

     nb_model.fit(X_train, y_train)
```

```
[9]: MultinomialNB
     ▶ Parameters
```

Step 10: Make Predictions

```
[10]: nb_predictions = nb_model.predict(X_test)
```

Step 11: Evaluate Naïve Bayes Model

```
[11]: nb_accuracy = accuracy_score(y_test, nb_predictions)

      nb_precision = precision_score(y_test, nb_predictions)

      nb_recall = recall_score(y_test, nb_predictions)

      nb_f1 = f1_score(y_test, nb_predictions)

      print("Naïve Bayes Results")
      print("Accuracy :", nb_accuracy)
      print("Precision:", nb_precision)
      print("Recall   :", nb_recall)
      print("F1-Score :", nb_f1)

      print("\nClassification Report:")
      print(classification_report(y_test, nb_predictions))
```

```
Naïve Bayes Results
Accuracy : 0.968609865470852
Precision: 1.0
Recall   : 0.7666666666666667
F1-Score : 0.8679245283018868

Classification Report:
              precision    recall  f1-score   support

           0       0.96      1.00      0.98       965
           1       1.00      0.77      0.87       150

    accuracy                           0.97      1115
   macro avg       0.98      0.88      0.93      1115
weighted avg       0.97      0.97      0.97      1115
```

Part B: KNN Implementation

Step 12: Train KNN Model

```
[12]: knn_model = KNeighborsClassifier(n_neighbors=5)

      knn_model.fit(X_train, y_train)
```

```
[12]: KNeighborsClassifier
      ▶ Parameters
```

Step 13: Make Predictions

```
[13]: knn_predictions = knn_model.predict(X_test)
```

Step 14: Evaluate KNN Model

```
[14]: knn_accuracy = accuracy_score(y_test, knn_predictions)

      knn_precision = precision_score(y_test, knn_predictions)

      knn_recall = recall_score(y_test, knn_predictions)

      knn_f1 = f1_score(y_test, knn_predictions)

      print("KNN Results")
      print("Accuracy :", knn_accuracy)
      print("Precision:", knn_precision)
      print("Recall   :", knn_recall)
      print("F1-Score :", knn_f1)

      print("\nClassification Report:")
      print(classification_report(y_test, knn_predictions))
```

```
KNN Results
Accuracy : 0.9103139013452914
Precision: 1.0
Recall   : 0.3333333333333333
F1-Score : 0.5

Classification Report:
              precision    recall  f1-score   support

           0       0.91      1.00      0.95       965
           1       1.00      0.33      0.50       150

    accuracy                           0.91      1115
   macro avg       0.95      0.67      0.73      1115
weighted avg       0.92      0.91      0.89      1115
```

Comparison of Results

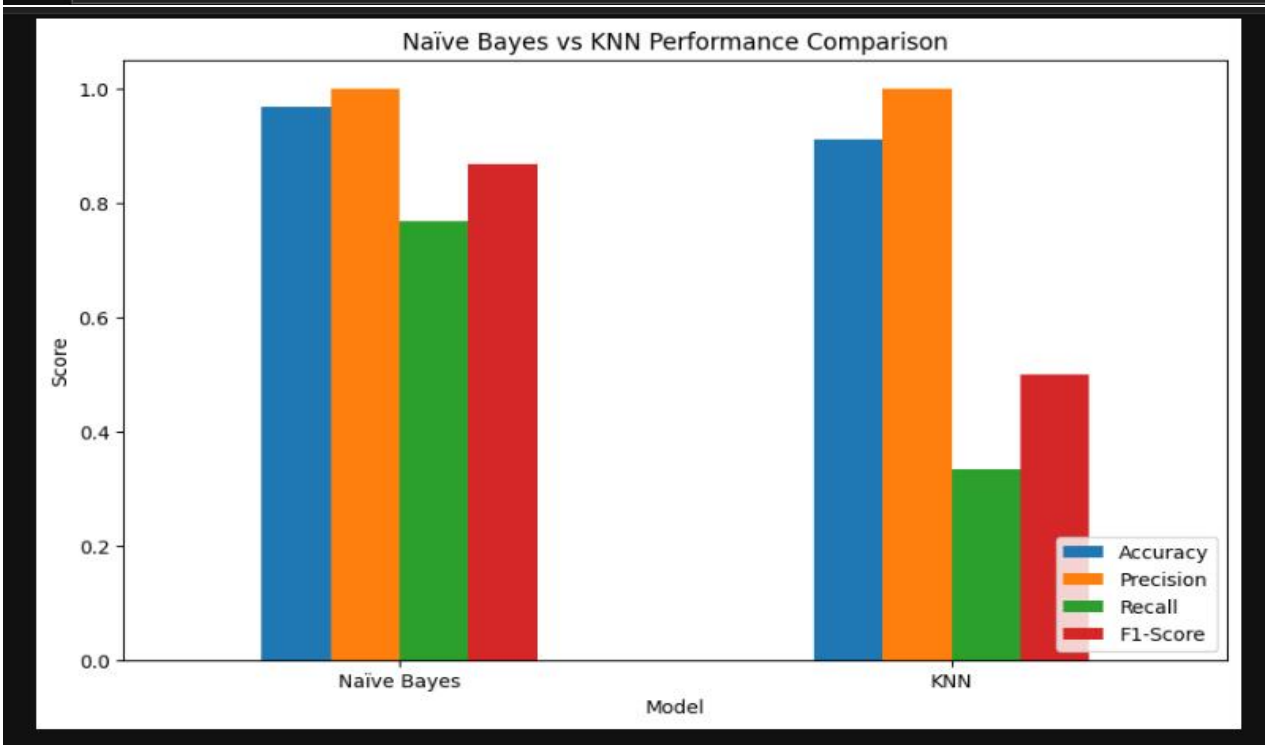
Step 15: Compare Both Models

```
[15]: comparison = pd.DataFrame({
          'Model': ['Naïve Bayes', 'KNN'],
          'Accuracy': [nb_accuracy, knn_accuracy],
          'Precision': [nb_precision, knn_precision],
          'Recall': [nb_recall, knn_recall],
          'F1-Score': [nb_f1, knn_f1]
      })

      print(comparison)
```

```
         Model  Accuracy  Precision    Recall  F1-Score
0  Naïve Bayes  0.968610        1.0  0.766667  0.867925
1          KNN  0.910314        1.0  0.333333  0.500000
```

Step 16: Visual Comparison

```
[16]: import matplotlib.pyplot as plt

      comparison.set_index('Model').plot(
          kind='bar',
          figsize=(10, 6)
      )

      plt.title('Naïve Bayes vs KNN Performance Comparison')

      plt.ylabel('Score')

      plt.xticks(rotation=0)

      plt.legend(loc='lower right')

      plt.show()
```



LAB-11

Credit Card Fraud Detection using Support Vector Machine (SVM)

Lab Title

Implementation of Support Vector Machine (Linear & RBF Kernel) for Credit Card Fraud Detection and Model Evaluation

Theory

Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised machine learning algorithm used for **classification** and **regression** problems. It aims to find the optimal hyperplane that separates different classes with the **maximum margin**.

Key Concepts in SVM

1. Hyperplane

A hyperplane is a decision boundary that separates different classes.

For two-dimensional data:

$$w^T x + b = 0$$

Where:

- **w** = Weight vector
- **x** = Feature vector
- **b** = Bias term

2. Support Vectors

Support vectors are the data points closest to the hyperplane. These points determine the position and orientation of the decision boundary.

3. Margin

Margin is the distance between the hyperplane and the nearest data points from each class.

SVM tries to maximize this margin to improve generalization.

Kernel Concepts

Sometimes data is **not linearly separable**. Kernels transform the data into higher-dimensional spaces where separation becomes easier.

1. Linear Kernel

Used when data is approximately linearly separable.

Formula:

$$K(x_i, x_j) = x_i^T x_j$$

2. Radial Basis Function (RBF) Kernel

The RBF kernel maps data into a higher-dimensional space to handle nonlinear relationships.

Formula:

$$K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$$

Where:

- **γ (gamma)** controls the influence of a single training example.

Dataset

Use the **Credit Card Fraud Detection Dataset** available on Kaggle.

Dataset Link:

Credit Card Fraud Detection Dataset

This dataset contains transactions made by European credit card holders.

Jupyter Notebook Code

Step 1: Import Libraries

```
[1]: import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt

     from sklearn.model_selection import train_test_split
     from sklearn.preprocessing import StandardScaler

     from sklearn.svm import SVC

     from sklearn.metrics import accuracy_score
     from sklearn.metrics import precision_score
     from sklearn.metrics import recall_score
     from sklearn.metrics import f1_score
     from sklearn.metrics import classification_report
     from sklearn.metrics import confusion_matrix
```

Step 2: Load Dataset

```
[2]: df = pd.read_csv('creditcard.csv')

     df.head()
```

[2]:

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.1
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.1
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.1
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.2
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.5

5 rows × 31 columns

Step 3: Explore Dataset

```
[3]: print(df.info())

     print(df['Class'].value_counts())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
```

Step 4: Handle Class Imbalance

```
[4]: fraud = df[df['Class'] == 1]

     normal = df[df['Class'] == 0]

     normal_sample = normal.sample(
         n=len(fraud),
         random_state=42
     )

     balanced_data = pd.concat([fraud, normal_sample])

     balanced_data = balanced_data.sample(
         frac=1,
         random_state=42
     )

     print(balanced_data['Class'].value_counts())
```

```
Class
0    492
1    492
Name: count, dtype: int64
```

Step 5: Define Features and Target Variable

```
[5]: X = balanced_data.drop('Class', axis=1)

     y = balanced_data['Class']
```

Step 6: Split Dataset

```
[6]: X_train, X_test, y_train, y_test = train_test_split(
         X,
         y,
         test_size=0.2,
         random_state=42,
         stratify=y
     )
```

Step 7: Feature Scaling

```
[7]: scaler = StandardScaler()

     X_train = scaler.fit_transform(X_train)

     X_test = scaler.transform(X_test)
```

Part A: SVM with Linear Kernel

Step 8: Train Linear SVM

```
[8]: linear_svm = SVC(
         kernel='linear',
         random_state=42
     )

     linear_svm.fit(X_train, y_train)
```

```
[8]: SVC
     ▶ Parameters
```

Step 9: Predictions

```
[9]: linear_pred = linear_svm.predict(X_test)
```

Step 10: Evaluate Linear SVM

```
[10]: linear_accuracy = accuracy_score(y_test, linear_pred)

      linear_precision = precision_score(y_test, linear_pred)

      linear_recall = recall_score(y_test, linear_pred)

      linear_f1 = f1_score(y_test, linear_pred)

      print("Linear Kernel SVM Results")

      print("Accuracy :", linear_accuracy)

      print("Precision:", linear_precision)

      print("Recall   :", linear_recall)

      print("F1-Score :", linear_f1)

      print("\nClassification Report:")

      print(classification_report(y_test, linear_pred))
```

```
Linear Kernel SVM Results
Accuracy : 0.9289340101522843
Precision: 0.9468085106382979
Recall   : 0.9081632653061225
F1-Score : 0.9270833333333334

Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.95      0.93        99
           1       0.95      0.91      0.93        98

    accuracy                           0.93       197
   macro avg       0.93      0.93      0.93       197
weighted avg       0.93      0.93      0.93       197
```

Part B: SVM with RBF Kernel

Step 11: Train RBF SVM

```
[11]: rbf_svm = SVC(
          kernel='rbf',
          gamma='scale',
          random_state=42
      )

      rbf_svm.fit(X_train, y_train)
```

```
[11]: SVC
      ▶ Parameters
```

Step 12: Predictions

```
[12]: rbf_pred = rbf_svm.predict(X_test)
```

Step 13: Evaluate RBF SVM

```
[13]: rbf_accuracy = accuracy_score(y_test, rbf_pred)

      rbf_precision = precision_score(y_test, rbf_pred)

      rbf_recall = recall_score(y_test, rbf_pred)

      rbf_f1 = f1_score(y_test, rbf_pred)

      print("RBF Kernel SVM Results")

      print("Accuracy :", rbf_accuracy)

      print("Precision:", rbf_precision)

      print("Recall   :", rbf_recall)

      print("F1-Score :", rbf_f1)

      print("\nClassification Report:")

      print(classification_report(y_test, rbf_pred))
```

```
RBF Kernel SVM Results
Accuracy : 0.9238578680203046
Precision: 0.9662921348314607
Recall   : 0.8775510204081632
F1-Score : 0.9197860962566845

Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.97      0.93        99
           1       0.97      0.88      0.92        98

    accuracy                           0.92       197
   macro avg       0.93      0.92      0.92       197
weighted avg       0.93      0.92      0.92       197
```

Confusion Matrix

Linear Kernel

```
[14]: cm_linear = confusion_matrix(y_test, linear_pred)

      print(cm_linear)

      [[94  5]
       [ 9 89]]
```

RBF Kernel

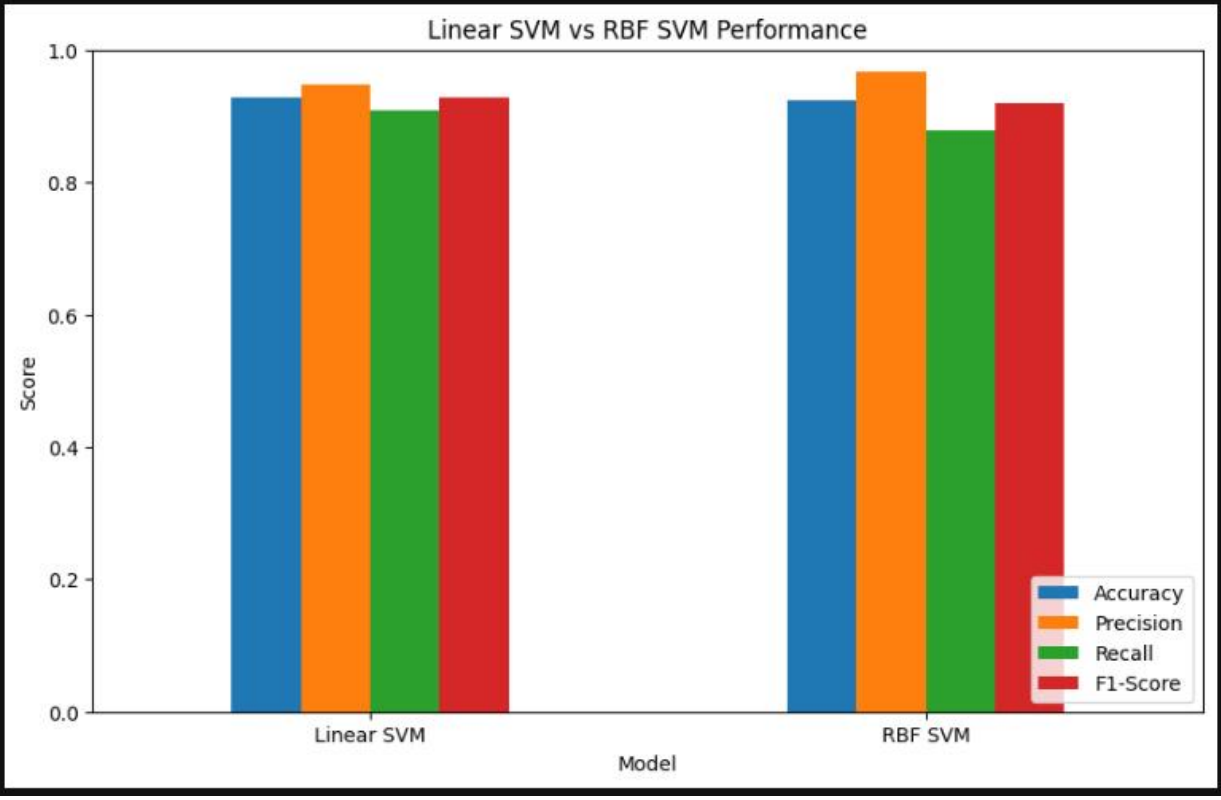
```
[15]: cm_rbf = confusion_matrix(y_test, rbf_pred)

      print(cm_rbf)

      [[96  3]
       [12 86]]
```

Model Comparison

```
[16]: comparison = pd.DataFrame({
          'Model': ['Linear SVM', 'RBF SVM'],
          'Accuracy': [linear_accuracy, rbf_accuracy],
          'Precision': [linear_precision, rbf_precision],
          'Recall': [linear_recall, rbf_recall],
          'F1-Score': [linear_f1, rbf_f1]
      })

      print(comparison)

          Model  Accuracy  Precision    Recall  F1-Score
      0  Linear SVM  0.928934   0.946809  0.908163  0.927083
      1     RBF SVM  0.923858   0.966292  0.877551  0.919786
```

Visual Comparison

```
[17]: comparison.set_index('Model').plot(
          kind='bar',
          figsize=(10,6)
      )

      plt.title('Linear SVM vs RBF SVM Performance')

      plt.ylabel('Score')

      plt.xticks(rotation=0)

      plt.ylim(0, 1)

      plt.legend(loc='lower right')

      plt.show()
```



LAB-12

Customer Segmentation using K-Means and Hierarchical Clustering

Lab Title

Implementation of K-Means Clustering and Hierarchical Clustering for Customer Segmentation.

Theory

Clustering

Clustering is an **unsupervised machine learning technique** used to group similar data points into clusters based on their characteristics.

Unlike supervised learning, clustering does not require labeled data.

1. K-Means Clustering

K-Means is a partition-based clustering algorithm that divides the dataset into **K clusters**.

Working of K-Means

1. Select the number of clusters (**K**).
2. Initialize K centroids randomly.
3. Assign each data point to the nearest centroid.
4. Recalculate centroids.
5. Repeat Steps 3 and 4 until centroids stop changing.

2. Hierarchical Clustering

Hierarchical clustering builds a hierarchy of clusters using a tree-like structure called a **Dendrogram**.

There are two approaches:

Agglomerative (Bottom-Up)

- Each data point starts as its own cluster.
- Closest clusters are merged repeatedly.

Divisive (Top-Down)

- Starts with one large cluster.

- Splits clusters recursively.

In this lab, we use **Agglomerative Clustering**.

Dataset

Dataset Name:

Shopping Mall Customer Segmentation Data

Since you already downloaded it from Kaggle, you can use the uploaded file directly.

Original Kaggle Link:

[Mall Customer Segmentation Dataset]

Jupyter Notebook Code

Step 1: Import Libraries

```
[1]: import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt

     from sklearn.cluster import KMeans
     from sklearn.cluster import AgglomerativeClustering
     from sklearn.preprocessing import StandardScaler

     from scipy.cluster.hierarchy import dendrogram, linkage
```

Step 2: Load Dataset

```
[2]: df = pd.read_csv("Shopping Mall Customer Segmentation Data .csv")

     df.head()
```

[2]:

	Customer ID	Age	Gender	Annual Income	Spending Score
0	d410ea53-6661-42a9-ad3a-f554b05fd2a7	30	Male	151479	89
1	1770b26f-493f-46b6-837f-4237fb5a314e	58	Female	185088	95
2	e81aa8eb-1767-4b77-87ce-1620dc732c5e	62	Female	70912	76
3	9795712a-ad19-47bf-8886-4f997d6046e3	23	Male	55460	57
4	64139426-2226-4cd6-bf09-91bce4b4db5e	24	Male	153752	76

Step 3: Explore Dataset

```
[3]: print(df.info())

     print(df.describe())

     print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15079 entries, 0 to 15078
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Customer ID     15079 non-null  object
 1   Age             15079 non-null  int64
 2   Gender          15079 non-null  object
 3   Annual Income   15079 non-null  int64
 4   Spending Score  15079 non-null  int64
dtypes: int64(3), object(2)
memory usage: 589.1+ KB
None
                Age  Annual Income  Spending Score
count  15079.000000   15079.000000    15079.000000
mean      54.191591  109742.880562       50.591617
std       21.119207   52249.425866       28.726977
min       18.000000   20022.000000        1.000000
25%       36.000000   64141.000000       26.000000
```

Step 4: Select Features

```
[8]: X = df[['Annual Income', 'Spending Score']]
```

Step 5: Feature Scaling

```
[9]: scaler = StandardScaler()

     X_scaled = scaler.fit_transform(X)
```

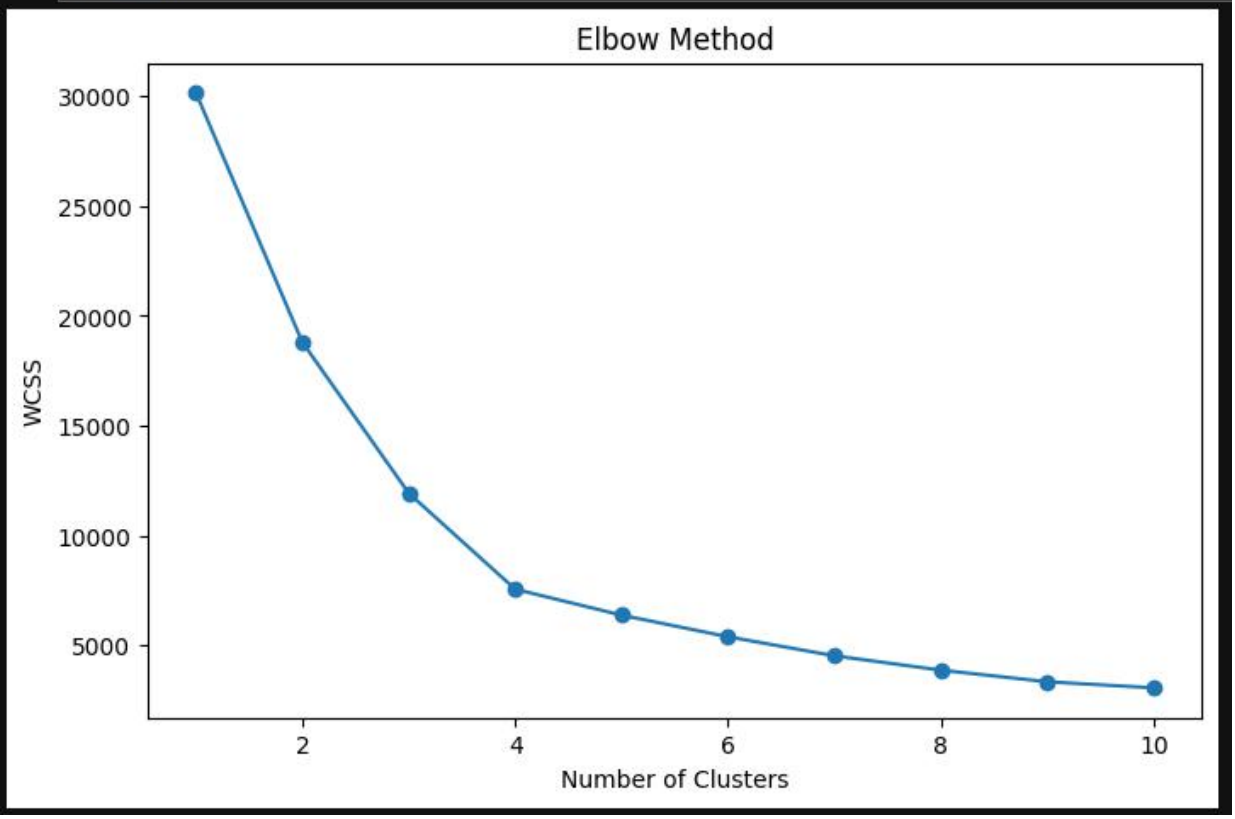
Part A: K-Means Clustering

Step 6: Find Optimal Number of Clusters (Elbow Method)

```
[10]: wcss = []

      for i in range(1, 11):

          kmeans = KMeans(
              n_clusters=i,
              init='k-means++',
              random_state=42,
              n_init=10
          )

          kmeans.fit(X_scaled)

          wcss.append(kmeans.inertia_)
```

Step 7: Visualize Elbow Method

```
[11]: plt.figure(figsize=(8,5))
      plt.plot(range(1,11), wcss, marker='o')
      plt.title('Elbow Method')
      plt.xlabel('Number of Clusters')
      plt.ylabel('WCSS')
      plt.show()
```

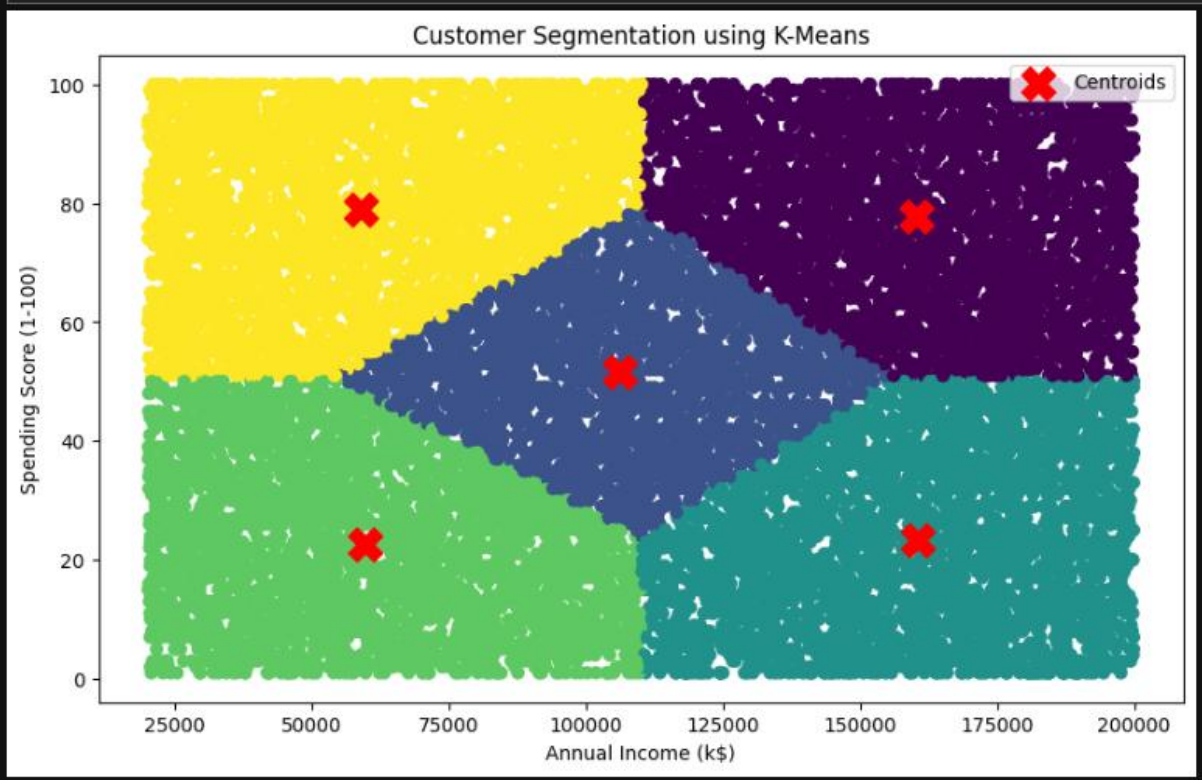


Step 8: Apply K-Means

```
[12]: kmeans = KMeans(
          n_clusters=5,
          init='k-means++',
          random_state=42,
          n_init=10
      )

      y_kmeans = kmeans.fit_predict(X_scaled)
```

Step 9: K-Means Cluster Visualization

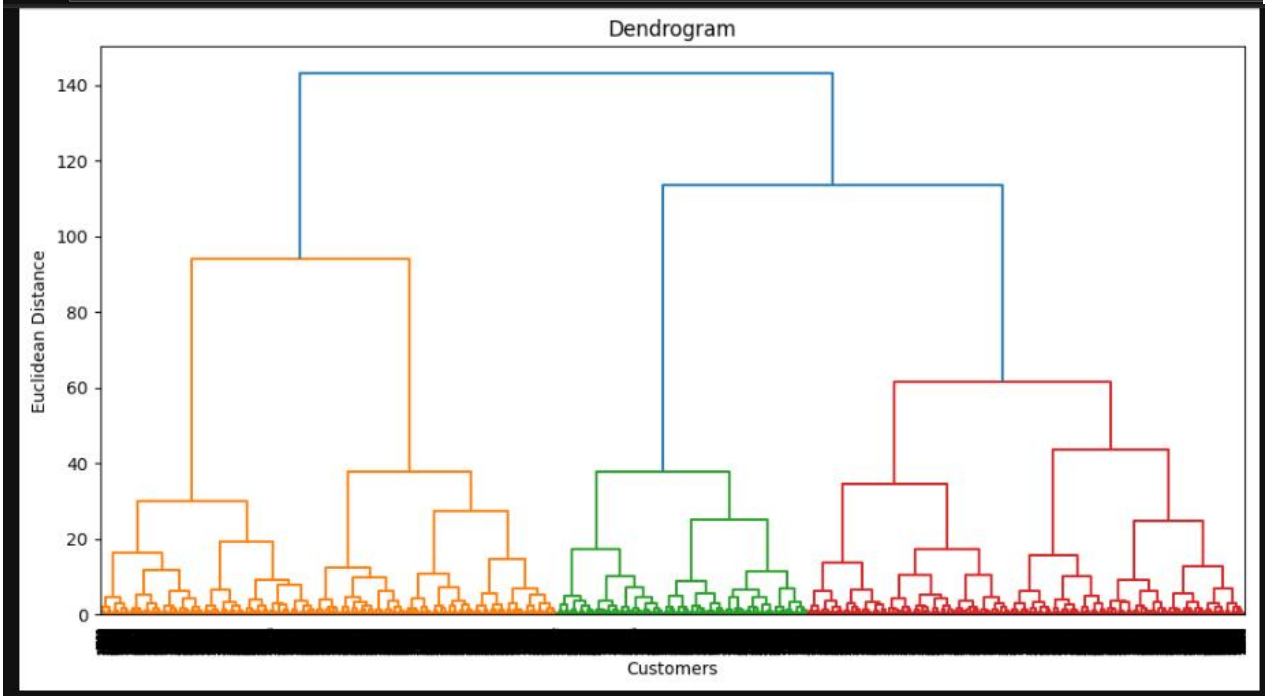
```
[13]: plt.figure(figsize=(10,6))

      plt.scatter(
          X.iloc[:,0],
          X.iloc[:,1],
          c=y_kmeans,
          cmap='viridis'
      )

      plt.scatter(
          scaler.inverse_transform(kmeans.cluster_centers_)[:,0],
          scaler.inverse_transform(kmeans.cluster_centers_)[:,1],
          s=300,
          c='red',
          marker='X',
          label='Centroids'
      )

      plt.title('Customer Segmentation using K-Means')

      plt.xlabel('Annual Income (k$)')

      plt.ylabel('Spending Score (1-100)')

      plt.legend()

      plt.show()
```



Part B: Hierarchical Clustering

Step 10: Create Dendrogram

```
[14]: plt.figure(figsize=(12,6))

      dendrogram(
          linkage(X_scaled, method='ward')
      )

      plt.title('Dendrogram')

      plt.xlabel('Customers')

      plt.ylabel('Euclidean Distance')

      plt.show()
```

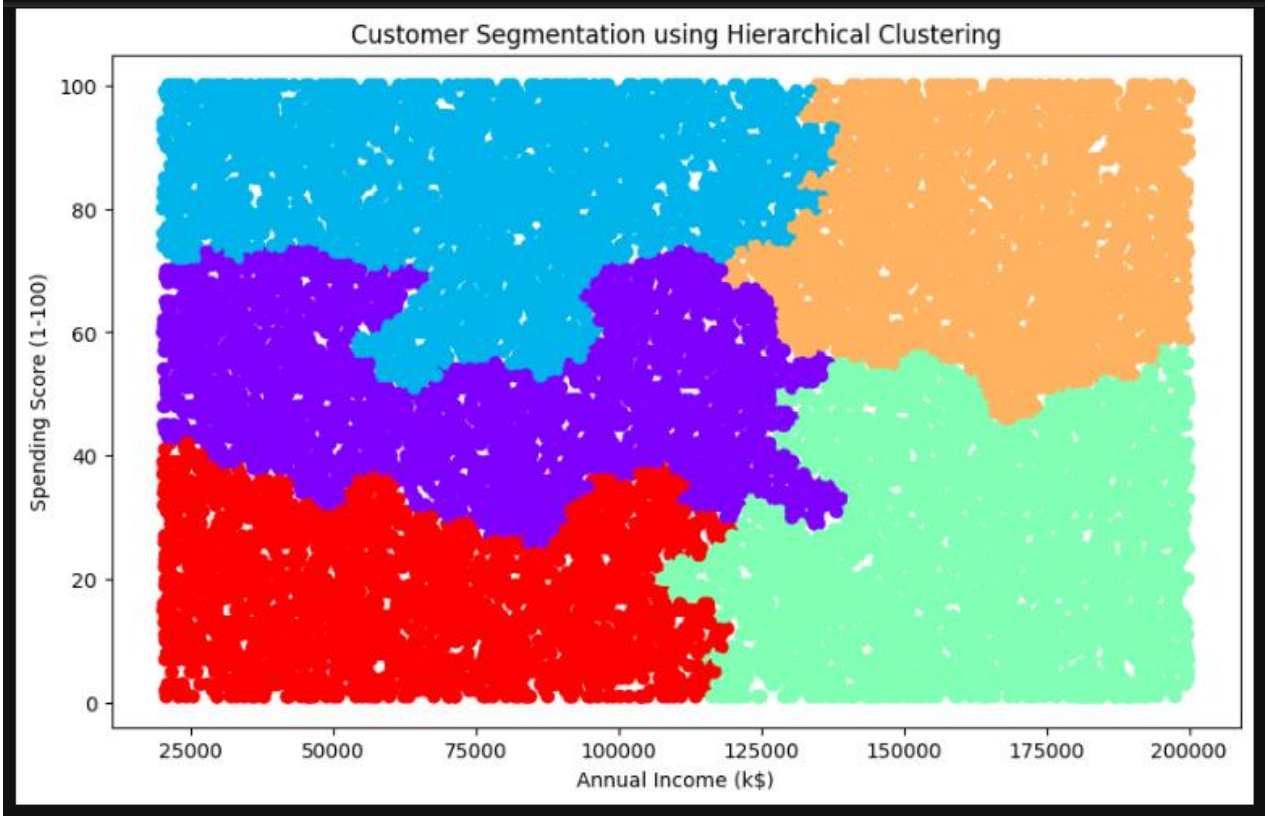


Step 11: Apply Hierarchical Clustering

```
[15]: hc = AgglomerativeClustering(
          n_clusters=5,
          metric='euclidean',
          linkage='ward'
      )

      y_hc = hc.fit_predict(X_scaled)
```

Step 12: Hierarchical Cluster Visualization

```
[16]: plt.figure(figsize=(10,6))

      plt.scatter(
          X.iloc[:,0],
          X.iloc[:,1],
          c=y_hc,
          cmap='rainbow'
      )

      plt.title('Customer Segmentation using Hierarchical Clustering')

      plt.xlabel('Annual Income (k$)')

      plt.ylabel('Spending Score (1-100)')

      plt.show()
```



Comparison of Algorithms

Feature	K-Means	Hierarchical Clustering
Type	Partition-Based	Tree-Based
Need Number of Clusters Initially	Yes	No
Speed	Fast	Slow

Feature	K-Means	Hierarchical Clustering
Suitable for Large Data	Yes	No
Dendrogram Visualization	No	Yes
Computational Complexity	Low	High

LAB-13

Anomaly Detection in Banking Transactions using Distance-Based Outliers and Isolation Forest

Lab Title

Implementation of Distance-Based Outlier Detection and Isolation Forest for Detecting Abnormal Banking Transactions

Theory

Anomaly Detection

Anomaly detection is the process of identifying unusual patterns or observations that significantly differ from normal behavior.

In banking systems, anomaly detection is used to identify:

- Fraudulent transactions
- Money laundering activities
- Unauthorized account access
- Suspicious transaction patterns

1. Distance-Based Outlier Detection

Distance-based methods identify anomalies by measuring the distance between data points.

Concept

An observation is considered an outlier if:

- It has very few neighboring points within a specified distance.
- It lies far away from the majority of the data.

The most commonly used distance-based algorithm is:

Local Outlier Factor (LOF)

LOF compares the local density of a point with the densities of its neighbors.

2. Isolation Forest

Isolation Forest is an ensemble-based anomaly detection algorithm.

Working Principle

Instead of profiling normal data points, Isolation Forest isolates anomalies.

Anomalies:

- Are few in number.
- Have attribute values very different from normal observations.

Therefore, they require fewer splits to isolate.

Dataset

Use the **Credit Card Fraud Detection Dataset** from Kaggle, which is widely used for abnormal banking transaction detection.

Dataset Link:

Credit Card Fraud Detection Dataset

Jupyter Notebook Code

Step 1: Import Libraries

```
[1]: import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt

     from sklearn.preprocessing import StandardScaler

     from sklearn.neighbors import LocalOutlierFactor
     from sklearn.ensemble import IsolationForest

     from sklearn.metrics import confusion_matrix
     from sklearn.metrics import classification_report
```

Step 2: Load Dataset

```
[2]: df = pd.read_csv("creditcard.csv")

     df.head()
```

[2]:

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.1
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.1
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.1
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.2
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.5

5 rows × 31 columns

Step 3: Explore Dataset

```
[3]: print(df.info())

     print(df['Class'].value_counts())

     print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
```

Step 4: Prepare Features and Labels

```
[4]: X = df.drop('Class', axis=1)

     y = df['Class']
```

Step 5: Feature Scaling

Scale the **Amount** and **Time** features.

```
[5]: scaler = StandardScaler()

     X_scaled = scaler.fit_transform(X)
```

Part A: Distance-Based Outlier Detection (LOF)

Step 6: Train LOF Model

```
[6]: lof = LocalOutlierFactor(
         n_neighbors=20,
         contamination=0.0017
     )

     lof_predictions = lof.fit_predict(X_scaled)
```

```
[7]: lof_predictions = np.where(
         lof_predictions == -1,
         1,
         0
     )
```

Step 7: Evaluate LOF

```
[8]: cm_lof = confusion_matrix(
         y,
         lof_predictions
     )

     print("LOF Confusion Matrix")

     print(cm_lof)

     print("\nClassification Report")

     print(classification_report(
         y,
         lof_predictions
     ))
```

```
LOF Confusion Matrix
[[283830    485]
 [   492      0]]

Classification Report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    284315
           1       0.00      0.00      0.00       492

    accuracy                           1.00    284807
   macro avg       0.50      0.50      0.50    284807
weighted avg       1.00      1.00      1.00    284807
```

Part B: Isolation Forest

Step 8: Train Isolation Forest

```
[9]: iso = IsolationForest(
         n_estimators=100,
         contamination=0.0017,
         random_state=42
     )

     iso.fit(X_scaled)

     iso_predictions = iso.predict(X_scaled)
```

```
[10]: iso_predictions = np.where(
          iso_predictions == -1,
          1,
          0
      )
```

Step 9: Evaluate Isolation Forest

```
[11]: cm_iso = confusion_matrix(
          y,
          iso_predictions
      )

      print("Isolation Forest Confusion Matrix")

      print(cm_iso)

      print("\nClassification Report")

      print(classification_report(
          y,
          iso_predictions
      ))
```

```
Isolation Forest Confusion Matrix
[[283955    360]
 [   367    125]]

Classification Report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    284315
           1       0.26      0.25      0.26       492

    accuracy                           1.00    284807
   macro avg       0.63      0.63      0.63    284807
weighted avg       1.00      1.00      1.00    284807
```

Anomaly Detection Rate (ADR)

ADR measures the proportion of actual anomalies correctly detected.

Formula:

ADR=\frac{TP}{TP+FN}

This is equivalent to **Recall**.

Calculate ADR

LOF

```
[12]: TN, FP, FN, TP = cm_lof.ravel()

      adr_lof = TP / (TP + FN)

      print("LOF Anomaly Detection Rate:", adr_lof)
```

```
LOF Anomaly Detection Rate: 0.0
```

Isolation Forest

```
[13]: TN, FP, FN, TP = cm_iso.ravel()

      adr_iso = TP / (TP + FN)

      print("Isolation Forest Anomaly Detection Rate:", adr_iso)

      Isolation Forest Anomaly Detection Rate: 0.2540650406504065
```

False Alarm Rate (FAR)

FAR measures the percentage of normal transactions incorrectly classified as anomalies.

Formula:

FAR=\frac{FP}{FP+TN}

Calculate FAR

LOF

```
[14]: TN, FP, FN, TP = cm_lof.ravel()

      far_lof = FP / (FP + TN)

      print("LOF False Alarm Rate:", far_lof)

      LOF False Alarm Rate: 0.0017058544220319013
```

Isolation Forest

```
[15]: TN, FP, FN, TP = cm_iso.ravel()

      far_iso = FP / (FP + TN)

      print("Isolation Forest False Alarm Rate:", far_iso)

      Isolation Forest False Alarm Rate: 0.0012662012204772875
```

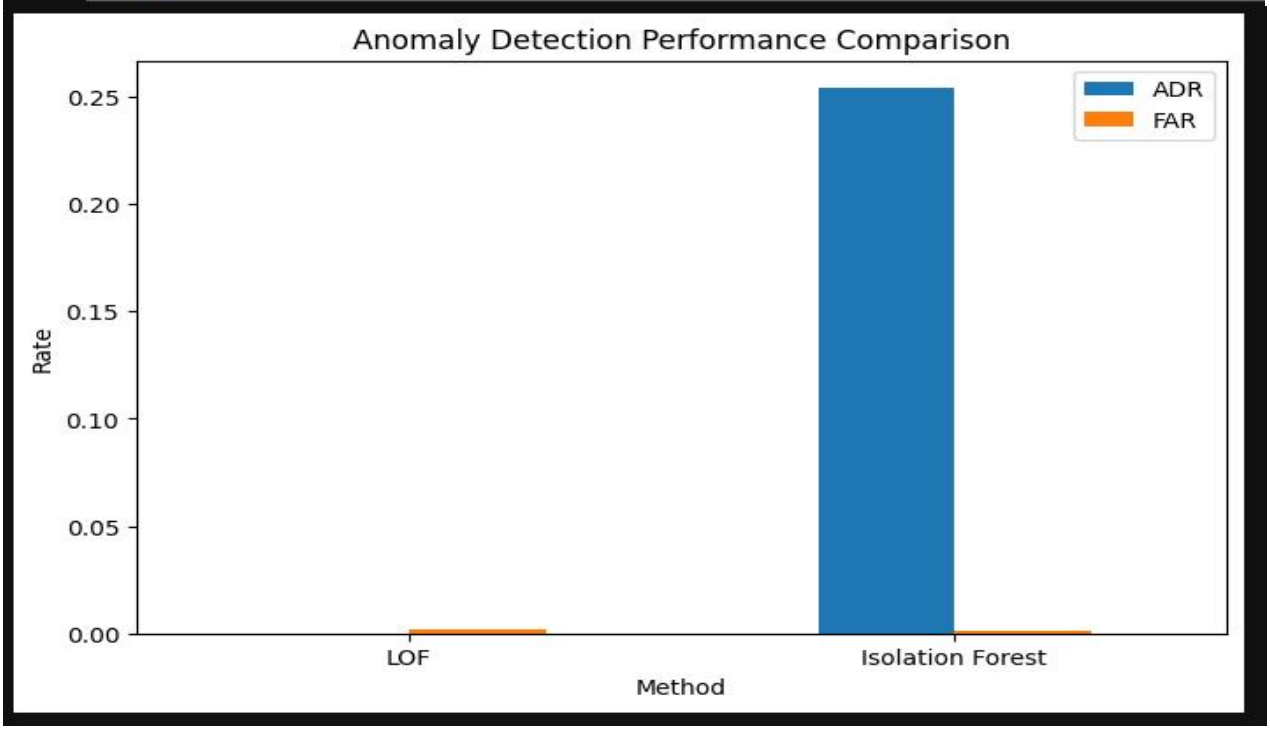
Comparison of Methods

```
[16]: comparison = pd.DataFrame({
          'Method': ['LOF', 'Isolation Forest'],
          'ADR': [adr_lof, adr_iso],
          'FAR': [far_lof, far_iso]
      })

      print(comparison)
```

```
              Method       ADR       FAR
0               LOF  0.000000  0.001706
1  Isolation Forest  0.254065  0.001266
```

Visualization of Results

```
[17]: comparison.set_index('Method').plot(
          kind='bar',
          figsize=(8,5)
      )

      plt.title(
          'Anomaly Detection Performance Comparison'
      )

      plt.ylabel('Rate')

      plt.xticks(rotation=0)

      plt.show()
```



LAB-14

Social Media Data Mining and Sentiment Analysis for Product Reviews

Lab Title

Data Scraping Basics, Sentiment Analysis, and Social Media Dataset Mining for Product Review Analysis

Theory

1. Data Scraping Basics

Data scraping (Web Scraping) is the process of automatically extracting data from websites.

In sentiment analysis, data scraping is used to collect:

- Product reviews from Amazon, Daraz, etc.
- Tweets from Twitter/X.
- Customer feedback from social media.
- Comments from YouTube or Facebook.

Common Python Libraries for Data Scraping

BeautifulSoup

Used to extract information from HTML pages.

```
from bs4 import BeautifulSoup
```

Requests

Used to send HTTP requests to websites.

```
import requests
```

Selenium

Used for scraping dynamic websites that load content using JavaScript.

```
from selenium import webdriver
```

Basic Web Scraping Example

```
[2]: import requests

     url = "https://books.toscrape.com/"

     response = requests.get(url)

     print(response.status_code)
```

```
200
```

```
[3]: from bs4 import BeautifulSoup

     soup = BeautifulSoup(response.text, 'html.parser')

     print(soup.prettify()[:1000])
```

```
<!DOCTYPE html>
<!--[if lt IE 7]>      <html lang="en-us" class="no-js lt-ie9 lt-ie8 lt-ie7"> <![endif]-->
<!--[if IE 7]>         <html lang="en-us" class="no-js lt-ie9 lt-ie8"> <![endif]-->
<!--[if IE 8]>         <html lang="en-us" class="no-js lt-ie9"> <![endif]-->
<!--[if gt IE 8]><!-->
<html class="no-js" lang="en-us">
 <!--<![endif]-->
 <head>
  <title>
   All products | Books to Scrape - Sandbox
  </title>
  <meta content="text/html; charset=utf-8" http-equiv="content-type"/>
  <meta content="24th Jun 2016 09:29" name="created"/>
  <meta content="" name="description"/>
  <meta content="width=device-width" name="viewport"/>
  <meta content="NOARCHIVE,NOCACHE" name="robots"/>
  <!-- Le HTML5 shim, for IE6-8 support of HTML elements -->
  <!--[if lt IE 9]>
        <script src="//html5shim.googlecode.com/svn/trunk/html5.js"></script>
```

```
[4]: import requests
     from bs4 import BeautifulSoup

     url = "https://books.toscrape.com/"

     response = requests.get(url)

     soup = BeautifulSoup(response.text, 'html.parser')

     books = soup.find_all('h3')

     for book in books:
         title = book.find('a')['title']
         print(title)
```

```
A Light in the Attic
Tipping the Velvet
Soumission
Sharp Objects
Sapiens: A Brief History of Humankind
The Requiem Red
```

Note: Always follow a website's Terms of Service before scraping data.

2. Sentiment Analysis

Sentiment Analysis is a Natural Language Processing (NLP) technique used to determine the emotional tone of text.

It classifies text into:

- **Positive Sentiment**
- **Negative Sentiment**
- **Neutral Sentiment**

TextBlob Sentiment Analysis

TextBlob provides polarity scores.

Polarity Range

Polarity Score	Sentiment
> 0	Positive
< 0	Negative
= 0	Neutral

3. Social Media Dataset Mining

Social media mining involves extracting useful knowledge from social media data.

Examples include:

- Understanding customer opinions.
- Detecting trends.
- Measuring brand reputation.
- Identifying consumer preferences.

Dataset

For product review sentiment analysis, use the

Dataset Link:

Use a smaller dataset:

Women's E-Commerce Clothing Reviews Dataset

Jupyter Notebook Code

Step 1: Install Required Libraries

```
[1]: !pip install textblob

Collecting textblob
  Downloading textblob-0.20.0-py3-none-any.whl.metadata (4.0 kB)
Collecting nltk>=3.9 (from textblob)
  Downloading nltk-3.9.4-py3-none-any.whl.metadata (3.2 kB)
Collecting click (from nltk>=3.9->textblob)
  Downloading click-8.4.1-py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: joblib in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from nltk>=3.9->textblob) (1.5.3)
Collecting regex>=2021.8.3 (from nltk>=3.9->textblob)
  Downloading regex-2026.5.9-cp310-cp310-win_amd64.whl.metadata (41 kB)
Requirement already satisfied: tqdm in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from nltk>=3.9->textblob) (4.67.3)
Requirement already satisfied: colorama in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from click->nltk>=3.9->textblob) (0.4.6)
Downloading textblob-0.20.0-py3-none-any.whl (624 kB)
   ---------------------------------------- 0.0/625.0 kB ? eta -:--:--
   ---------------------------------------- 0.0/625.0 kB ? eta -:--:--
   ---------------------------------- ------ 524.3/625.0 kB 1.7 MB/s eta 0:00:01
   ---------------------------------- ------ 524.3/625.0 kB 1.7 MB/s eta 0:00:01
   ---------------------------------------- 625.0/625.0 kB 828.4 kB/s  0:00:00
Downloading nltk-3.9.4-py3-none-any.whl (1.6 MB)
```

```
[2]: !pip install wordcloud

Collecting wordcloud
  Downloading wordcloud-1.9.6-cp310-cp310-win_amd64.whl.metadata (3.5 kB)
Requirement already satisfied: numpy>=1.19.3 in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from wordcloud) (2.0.1)
Requirement already satisfied: pillow in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from wordcloud) (11.1.0)
Requirement already satisfied: matplotlib in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from wordcloud) (3.10.8)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from matplotlib->wordcloud) (1.3.2)
Requirement already satisfied: cycler>=0.10 in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from matplotlib->wordcloud) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from matplotlib->wordcloud) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from matplotlib->wordcloud) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from matplotlib->wordcloud) (25.0)
Requirement already satisfied: pyparsing>=3 in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from matplotlib->wordcloud) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from matplotlib->wordcloud) (2.9.0.post
0)
Requirement already satisfied: six>=1.5 in c:\users\star\anaconda3\envs\torch_ok\lib\site-packages (from python-dateutil>=2.7->matplotlib->wordcloud)
(1.17.0)
Downloading wordcloud-1.9.6-cp310-cp310-win_amd64.whl (306 kB)
Installing collected packages: wordcloud
Successfully installed wordcloud-1.9.6
```

```
[3]: import nltk
     nltk.download('punkt')

     [nltk_data] Downloading package punkt to
     [nltk_data]     C:\Users\star\AppData\Roaming\nltk_data...
     [nltk_data]   Package punkt is already up-to-date!
[3]: True
```

Step 2: Import Libraries

```
[4]: import pandas as pd
     import numpy as np

     import matplotlib.pyplot as plt
     import seaborn as sns

     from textblob import TextBlob

     from sklearn.model_selection import train_test_split
     from sklearn.feature_extraction.text import TfidfVectorizer
     from sklearn.naive_bayes import MultinomialNB

     from sklearn.metrics import accuracy_score
     from sklearn.metrics import classification_report
     from sklearn.metrics import confusion_matrix
```

Step 3: Load Dataset

```
[5]: df = pd.read_csv("Womens Clothing E-Commerce Reviews.csv")

     df.head()
     print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23486 entries, 0 to 23485
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Unnamed: 0               23486 non-null  int64
 1   Clothing ID              23486 non-null  int64
 2   Age                      23486 non-null  int64
 3   Title                    19676 non-null  object
 4   Review Text              22641 non-null  object
 5   Rating                   23486 non-null  int64
 6   Recommended IND          23486 non-null  int64
 7   Positive Feedback Count  23486 non-null  int64
 8   Division Name            23472 non-null  object
 9   Department Name          23472 non-null  object
 10  Class Name               23472 non-null  object
dtypes: int64(6), object(5)
memory usage: 2.0+ MB
None
```

Step 4: Select Required Columns

```
[6]: df = df[['Review Text', 'Rating']]

     df.head()
```

[6]:

	Review Text	Rating
0	Absolutely wonderful - silky and sexy and comf...	4
1	Love this dress! it's sooo pretty. i happene...	5
2	I had such high hopes for this dress and reall...	3
3	I love, love, love this jumpsuit. it's fun, fl...	5
4	This shirt is very flattering to all due to th...	5

```
[7]: print(df.isnull().sum())
```

```
Review Text    845
Rating           0
dtype: int64
```

```
[8]: df.dropna(inplace=True)

     print(df.shape)
```

```
(22641, 2)
```

Step 5: Convert Ratings into Sentiments

```
[9]: def get_sentiment(rating):

         if rating >= 4:
             return 'Positive'

         elif rating <= 2:
             return 'Negative'

         else:
             return 'Neutral'
```

Apply:

```
[10]: df['Sentiment'] = df['Rating'].apply(get_sentiment)

      df.head()
```

[10]:

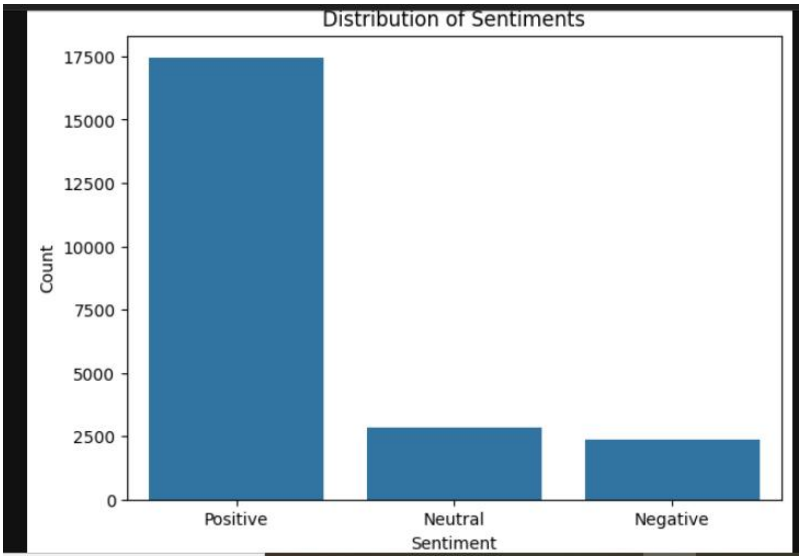
	Review Text	Rating	Sentiment
0	Absolutely wonderful - silky and sexy and comf...	4	Positive
1	Love this dress! it's sooo pretty. i happene...	5	Positive
2	I had such high hopes for this dress and reall...	3	Neutral
3	I love, love, love this jumpsuit. it's fun, fl...	5	Positive
4	This shirt is very flattering to all due to th...	5	Positive

Step 6: Check Sentiment Distribution

```
[11]: print(df['Sentiment'].value_counts())

      Sentiment
      Positive    17448
      Neutral      2823
      Negative     2370
      Name: count, dtype: int64
```

Visualization:

```
[12]: plt.figure(figsize=(7,5))

      sns.countplot(
          x='Sentiment',
          data=df
      )

      plt.title("Distribution of Sentiments")

      plt.xlabel("Sentiment")

      plt.ylabel("Count")

      plt.show()
```



Step 7: TextBlob Sentiment Analysis

```
[13]: df['Polarity'] = df['Review Text'].apply(
          lambda x: TextBlob(str(x)).sentiment.polarity
      )

      df[['Review Text', 'Polarity']].head()
```

[13]:

	Review Text	Polarity
0	Absolutely wonderful - silky and sexy and comf...	0.633333
1	Love this dress! it's sooo pretty. i happene...	0.339583
2	I had such high hopes for this dress and reall...	0.073675
3	I love, love, love this jumpsuit. it's fun, fl...	0.550000
4	This shirt is very flattering to all due to th...	0.512891

Step 8: Predict Sentiment Using Polarity

```
[14]: def predict_sentiment(polarity):

          if polarity > 0:
              return 'Positive'

          elif polarity < 0:
              return 'Negative'

          else:
              return 'Neutral'
```

Apply:

```
[15]: df['Predicted Sentiment'] = df['Polarity'].apply(
          predict_sentiment
      )

      df.head()
```

[15]:

	Review Text	Rating	Sentiment	Polarity	Predicted Sentiment
0	Absolutely wonderful - silky and sexy and comf...	4	Positive	0.633333	Positive
1	Love this dress! it's sooo pretty. i happene...	5	Positive	0.339583	Positive
2	I had such high hopes for this dress and reall...	3	Neutral	0.073675	Positive
3	I love, love, love this jumpsuit. it's fun, fl...	5	Positive	0.550000	Positive
4	This shirt is very flattering to all due to th...	5	Positive	0.512891	Positive

Step 9: Display Results

```
[16]: df[['Review Text',
          'Rating',
          'Sentiment',
          'Predicted Sentiment',
          'Polarity']].head(10)
```

[16]:

	Review Text	Rating	Sentiment	Predicted Sentiment	Polarity
0	Absolutely wonderful - silky and sexy and comf...	4	Positive	Positive	0.633333
1	Love this dress! it's sooo pretty. i happene...	5	Positive	Positive	0.339583
2	I had such high hopes for this dress and reall...	3	Neutral	Positive	0.073675
3	I love, love, love this jumpsuit. it's fun, fl...	5	Positive	Positive	0.550000
4	This shirt is very flattering to all due to th...	5	Positive	Positive	0.512891
5	I love tracy reese dresses, but this one is no...	2	Negative	Positive	0.178750
6	I aded this in my basket at hte last mintue to...	5	Positive	Positive	0.133750
7	I ordered this in carbon for store pick up, an...	4	Positive	Positive	0.171635
8	I love this dress. i usually get an xs but it ...	5	Positive	Positive	0.002500
9	I'm 5"5' and 125 lbs. i ordered the s petite t...	5	Positive	Positive	0.204200

Machine Learning Based Sentiment Analysis

Step 10: Remove Neutral Reviews

```
[17]: df_ml = df[
          df['Sentiment'] != 'Neutral'
      ]

      print(df_ml['Sentiment'].value_counts())
```

```
Sentiment
Positive    17448
Negative     2370
Name: count, dtype: int64
```

Step 11: Convert Labels to Numeric

```
[18]: df_ml['Sentiment'] = df_ml['Sentiment'].map({
          'Positive': 1,
          'Negative': 0
      })

      df_ml.head()
```

```
C:\Users\star\AppData\Local\Temp\ipykernel_1080\2897532642.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_ml['Sentiment'] = df_ml['Sentiment'].map({
```

[18]:

	Review Text	Rating	Sentiment	Polarity	Predicted Sentiment
0	Absolutely wonderful - silky and sexy and comf...	4	1	0.633333	Positive
1	Love this dress! it's sooo pretty. i happene...	5	1	0.339583	Positive
3	I love, love, love this jumpsuit. it's fun, fl...	5	1	0.550000	Positive
4	This shirt is very flattering to all due to th...	5	1	0.512891	Positive
5	I love tracy reese dresses, but this one is no...	2	0	0.178750	Positive

Step 12: Define Features and Labels

```
[19]: X = df_ml['Review Text']

      y = df_ml['Sentiment']
```

Step 13: TF-IDF Vectorization

```
[20]: tfidf = TfidfVectorizer(
          stop_words='english',
          max_features=5000
      )

      X_features = tfidf.fit_transform(X)

      print(X_features.shape)
```

```
(19818, 5000)
```

Step 14: Train-Test Split

```
[21]: X_train, X_test, y_train, y_test = train_test_split(
          X_features,
          y,
          test_size=0.2,
          random_state=42
      )
```

Step 15: Train Naive Bayes Model

```
[22]: model = MultinomialNB()

      model.fit(
          X_train,
          y_train
      )
```

```
[22]: MultinomialNB
      ▶ Parameters
```

Step 16: Make Predictions

```
[23]: y_pred = model.predict(X_test)

      print(y_pred[:10])
```

```
[1 1 1 1 1 1 1 1 1 1]
```

Step 17: Evaluate Model

```
[24]: accuracy = accuracy_score(
          y_test,
          y_pred
      )

      print("Accuracy:", accuracy)
```

```
Accuracy: 0.8920282542885973
```

```
[25]: print(classification_report(
          y_test,
          y_pred
      ))
```

```
              precision    recall  f1-score   support

           0       0.93      0.11      0.20       478
           1       0.89      1.00      0.94      3486

    accuracy                           0.89      3964
   macro avg       0.91      0.56      0.57      3964
weighted avg       0.90      0.89      0.85      3964
```

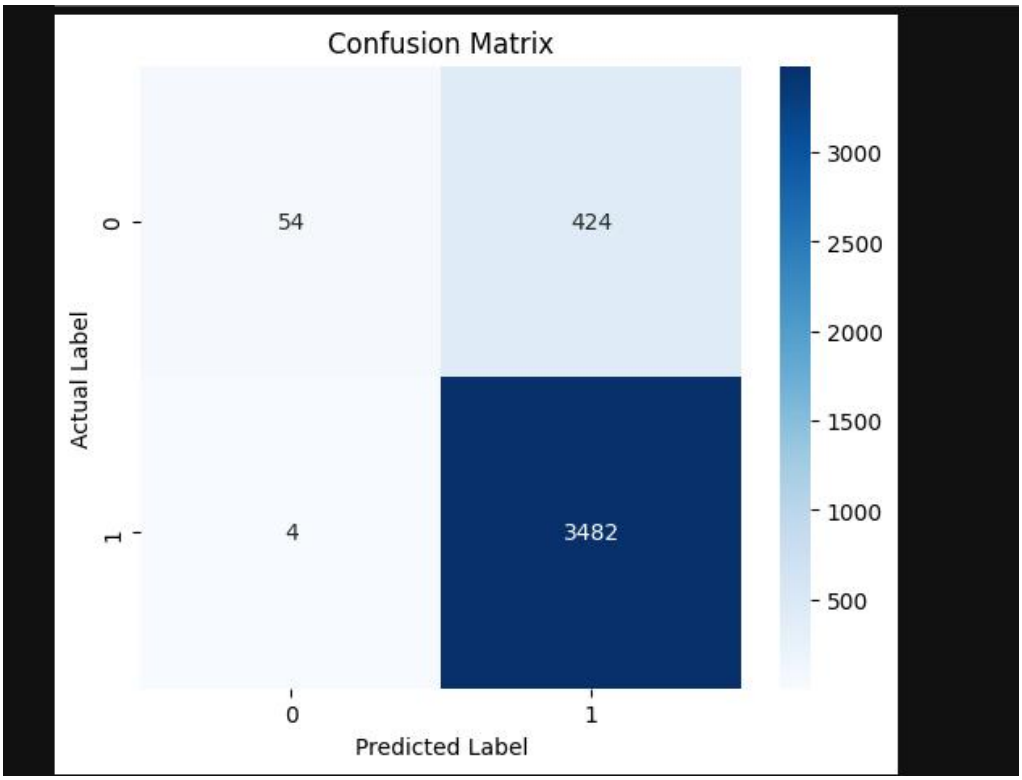
Step 18: Confusion Matrix

```
[26]: cm = confusion_matrix(
          y_test,
          y_pred
      )

      print(cm)
```

```
[[  54  424]
 [   4 3482]]
```

Visualization:

```
[27]: plt.figure(figsize=(6,5))

      sns.heatmap(
          cm,
          annot=True,
          fmt='d',
          cmap='Blues'
      )

      plt.xlabel("Predicted Label")

      plt.ylabel("Actual Label")

      plt.title("Confusion Matrix")

      plt.show()
```



Testing on New Reviews

[28]:
```
new_review = [
    "The dress quality is amazing and fits perfectly."
]

review_vector = tfidf.transform(
    new_review
)

prediction = model.predict(
    review_vector
)

if prediction[0] == 1:
    print("Positive Review")
else:
    print("Negative Review")
```

```
Positive Review
```

LAB-15

Comparative Analysis of Multiple Data Mining Models for Industry-Based Prediction

Lab Title

Implementation, Comparison, and Interpretation of Multiple Data Mining Models for Industry-Based Customer Response Prediction

Industry-Based Dataset

For this lab, use the **Bank Marketing Dataset**.

Dataset Link:

Bank Marketing Dataset (Kaggle)

Problem Statement

Predict whether a customer will subscribe to a **term deposit** after a marketing campaign.

Target Variable:

- **yes → 1**
- **no → 0**

This is a real-world banking industry classification problem

Theory

Models Used

Model	Type
Logistic Regression	Classification
Decision Tree	Classification
Random Forest	Ensemble Learning
Naïve Bayes	Probabilistic
K-Nearest Neighbors (KNN)	Distance-Based

Model	Type
Support Vector Machine (SVM)	Margin-Based
Gradient Boosting	Ensemble Learning
AdaBoost	Ensemble Learning

Why Compare Multiple Models?

Different algorithms have different strengths:

- Some perform better on linear relationships.
- Some capture complex patterns.
- Ensemble methods generally improve accuracy.
- Comparing models helps identify the most suitable solution.

Jupyter Notebook Code

Step 1: Import Libraries

```
[1]: import pandas as pd
     import numpy as np

     import matplotlib.pyplot as plt
     import seaborn as sns

     from sklearn.model_selection import train_test_split

     from sklearn.preprocessing import LabelEncoder
     from sklearn.preprocessing import StandardScaler

     from sklearn.metrics import accuracy_score
     from sklearn.metrics import classification_report

     from sklearn.linear_model import LogisticRegression

     from sklearn.tree import DecisionTreeClassifier

     from sklearn.ensemble import RandomForestClassifier
     from sklearn.ensemble import GradientBoostingClassifier
     from sklearn.ensemble import AdaBoostClassifier

     from sklearn.naive_bayes import GaussianNB

     from sklearn.neighbors import KNeighborsClassifier

     from sklearn.svm import SVC
```

Step 2: Load Dataset

```
[5]: df = pd.read_csv("bank.csv")

     df.head()
```

[5]:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	deposit
0	59	admin.	married	secondary	no	2343	yes	no	unknown	5	may	1042	1	-1	0	unknown	yes
1	56	admin.	married	secondary	no	45	no	no	unknown	5	may	1467	1	-1	0	unknown	yes
2	41	technician	married	secondary	no	1270	yes	no	unknown	5	may	1389	1	-1	0	unknown	yes
3	55	services	married	secondary	no	2476	yes	no	unknown	5	may	579	1	-1	0	unknown	yes
4	54	admin.	married	tertiary	no	184	no	no	unknown	5	may	673	2	-1	0	unknown	yes

Step 3: Explore Dataset

```
[6]: print(df.info())

     print("\nMissing Values:")
     print(df.isnull().sum())

     print("\nTarget Variable Distribution:")
     print(df['deposit'].value_counts())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11162 entries, 0 to 11161
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        11162 non-null  int64
 1   job        11162 non-null  object
 2   marital    11162 non-null  object
 3   education  11162 non-null  object
 4   default    11162 non-null  object
 5   balance    11162 non-null  int64
 6   housing    11162 non-null  object
 7   loan       11162 non-null  object
 8   contact    11162 non-null  object
 9   day        11162 non-null  int64
 10  month      11162 non-null  object
 11  duration   11162 non-null  int64
```

Step 4: Data Preprocessing

Convert Categorical Variables

```
[7]: le = LabelEncoder()

     for col in df.columns:
         if df[col].dtype == 'object':
             df[col] = le.fit_transform(df[col])

     df.head()
```

[7]:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	deposit
0	59	0	1	1	0	2343	1	0	2	5	8	1042	1	-1	0	3	1
1	56	0	1	1	0	45	0	0	2	5	8	1467	1	-1	0	3	1
2	41	9	1	1	0	1270	1	0	2	5	8	1389	1	-1	0	3	1
3	55	7	1	1	0	2476	1	0	2	5	8	579	1	-1	0	3	1
4	54	0	1	2	0	184	0	0	2	5	8	673	2	-1	0	3	1

Define Features and Target

```
[8]: X = df.drop('deposit', axis=1)

     y = df['deposit']
```

Train-Test Split

```
[9]: X_train, X_test, y_train, y_test = train_test_split(
         X,
         y,
         test_size=0.2,
         random_state=42
     )
```

Feature Scaling

```
[10]: scaler = StandardScaler()

      X_train = scaler.fit_transform(X_train)

      X_test = scaler.transform(X_test)
```

Step 5: Define Models

```
[11]: models = {

          "Logistic Regression":
          LogisticRegression(max_iter=1000),

          "Decision Tree":
          DecisionTreeClassifier(random_state=42),

          "Random Forest":
          RandomForestClassifier(random_state=42),

          "Naive Bayes":
          GaussianNB(),

          "KNN":
          KNeighborsClassifier(),

          "SVM":
          SVC(),

          "Gradient Boosting":
          GradientBoostingClassifier(random_state=42),

          "AdaBoost":
          AdaBoostClassifier(random_state=42)
      }
```

Step 6: Train and Evaluate Models

```
[12]: results = []

      for name, model in models.items():

          model.fit(X_train, y_train)

          y_pred = model.predict(X_test)

          accuracy = accuracy_score(y_test, y_pred)

          results.append({
              'Model': name,
              'Accuracy': accuracy
          })

          print("\n" + "="*50)

          print("Model:", name)

          print("Accuracy:", round(accuracy, 4))

          print("\nClassification Report:")

          print(classification_report(y_test, y_pred))
```

```
==================================================
Model: Logistic Regression
Accuracy: 0.79

Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.82      0.80      1166
           1       0.79      0.76      0.78      1067

    accuracy                           0.79      2233
   macro avg       0.79      0.79      0.79      2233
weighted avg       0.79      0.79      0.79      2233
```

```
==================================================
Model: Decision Tree
Accuracy: 0.7631

Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.78      0.78      1166
           1       0.76      0.74      0.75      1067

    accuracy                           0.76      2233
   macro avg       0.76      0.76      0.76      2233
weighted avg       0.76      0.76      0.76      2233
```

```
==================================================
Model: Random Forest
Accuracy: 0.8334

Classification Report:
              precision    recall  f1-score   support

           0       0.86      0.81      0.84      1166
           1       0.81      0.85      0.83      1067

    accuracy                           0.83      2233
   macro avg       0.83      0.83      0.83      2233
weighted avg       0.83      0.83      0.83      2233
```

```
==================================================
Model: Naive Bayes
Accuracy: 0.7465

Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.69      0.74      1166
           1       0.70      0.81      0.75      1067

    accuracy                           0.75      2233
   macro avg       0.75      0.75      0.75      2233
weighted avg       0.75      0.75      0.75      2233
```

```
================================================
Model: KNN
Accuracy: 0.7734

Classification Report:
              precision    recall  f1-score   support

           0       0.76      0.82      0.79      1166
           1       0.79      0.72      0.75      1067

    accuracy                           0.77      2233
   macro avg       0.78      0.77      0.77      2233
weighted avg       0.77      0.77      0.77      2233
```

```
================================================
Model: SVM
Accuracy: 0.8133

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.80      0.82      1166
           1       0.79      0.83      0.81      1067

    accuracy                           0.81      2233
   macro avg       0.81      0.81      0.81      2233
weighted avg       0.81      0.81      0.81      2233
```

```
================================================
Model: Gradient Boosting
Accuracy: 0.8222

Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.81      0.83      1166
           1       0.80      0.84      0.82      1067

    accuracy                           0.82      2233
   macro avg       0.82      0.82      0.82      2233
weighted avg       0.82      0.82      0.82      2233
```

```
================================================
Model: AdaBoost
Accuracy: 0.7967

Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.80      0.80      1166
           1       0.78      0.79      0.79      1067

    accuracy                           0.80      2233
   macro avg       0.80      0.80      0.80      2233
weighted avg       0.80      0.80      0.80      2233
```

Step 7: Create Comparison Table

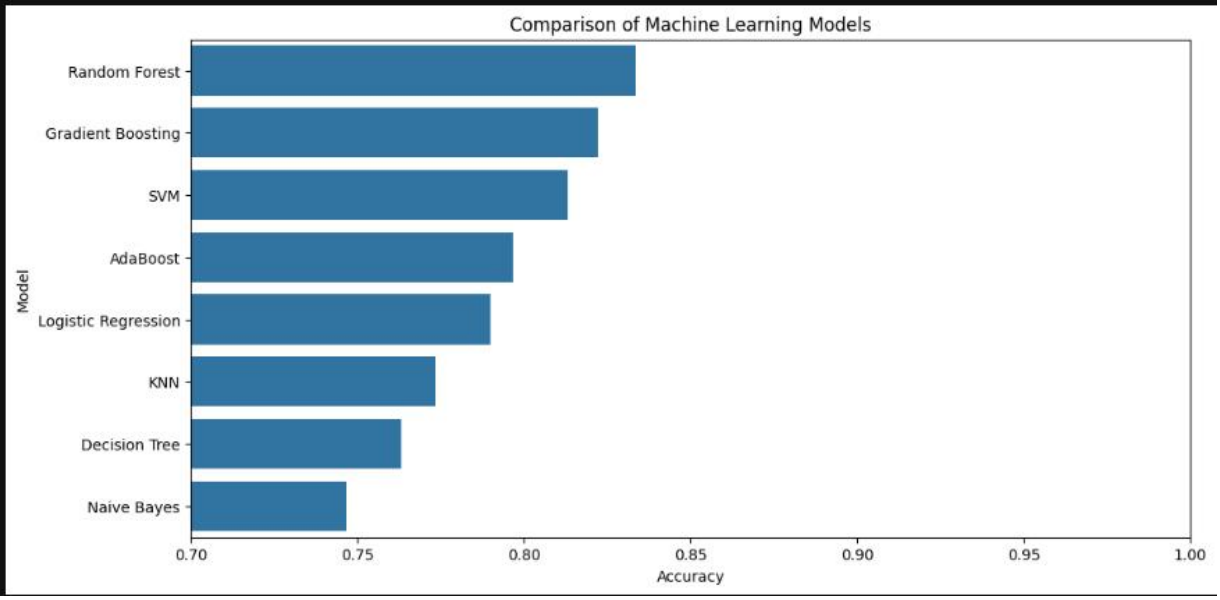
```
[16]: comparison = pd.DataFrame(results)

      comparison = comparison.sort_values(
          by='Accuracy',
          ascending=False
      )

      comparison
```

[16]:

	Model	Accuracy
2	Random Forest	0.833408
6	Gradient Boosting	0.822212
5	SVM	0.813256
7	AdaBoost	0.796686
0	Logistic Regression	0.789969
4	KNN	0.773399
1	Decision Tree	0.763099
3	Naive Bayes	0.746529

Step 8: Model Comparison Visualization

```
[17]: plt.figure(figsize=(12, 6))

      sns.barplot(
          x='Accuracy',
          y='Model',
          data=comparison
      )

      plt.title('Comparison of Machine Learning Models')

      plt.xlim(0.7, 1.0)

      plt.show()
```



Step 9: Identify Best Model

```
[18]: best_model = comparison.iloc[0]

      print("Best Model:", best_model['Model'])

      print("Best Accuracy:",
            round(best_model['Accuracy'], 4))

      Best Model: Random Forest
      Best Accuracy: 0.8334
```

Step 10: Business Interpretation

```
[19]: print("\nBusiness Interpretation")

      print(
          f"The best performing model is "
          f"{best_model['Model']} "
          f"with an accuracy of "
          f"{round(best_model['Accuracy']*100,2)}%."
      )

      print("\nBusiness Recommendations:")

      print("1. Target customers predicted as likely to subscribe.")

      print("2. Reduce marketing costs by focusing on high-potential customers.")

      print("3. Integrate the model into CRM systems.")

      print("4. Continuously retrain the model with new campaign data.")

      print("5. Use model insights to improve future marketing strategies.")
```

```
Business Interpretation
The best performing model is Random Forest with an accuracy of 83.34%.

Business Recommendations:
1. Target customers predicted as likely to subscribe.
2. Reduce marketing costs by focusing on high-potential customers.
3. Integrate the model into CRM systems.
4. Continuously retrain the model with new campaign data.
5. Use model insights to improve future marketing strategies.
```